Project Report

on

ThermalODE: A Physics-Informed Simulator for Multi-GPU HPC Clusters

Submitted by

Project Members

Saket Sontakke 1032222648
Taha Kapadia 1032222679
Pranjal Vishwakarma 1032222756
Akshat Srivastava 1032222619

Under the Internal Guidance of

Dr. Pratvina Talele

Department of Computer Engineering and Technology
MIT World Peace University, Kothrud,
Pune 411 038, Maharashtra - India
2025-2026

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU
॥ विश्वशान्तिर्ध्रुवं ध्रुवा ॥

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

C E R T I F I C A T E

This is to certify that,

Saket Sontakke

Taha Kapadia

Pranjal Vishwakarma

Akshat Srivastava

of BTech. (Computer Science & Engineering) have completed their project titled "*ThermalODE: A Physics-Informed Simulator for Multi-GPU HPC Clusters*" and have submitted this Capstone Project Report towards fulfillment of the requirement for the Degree-Bachelor of Computer Science & Engineering (BTech-CSE) for the academic year 2025-2026

[Dr. Pratvina Talele]
Project Guide
DCET
MIT World Peace University, Pune

[Dr. Balaji M Patil]
Program Director
DCET
MIT World Peace University, Pune

Date: 09/05/2026

Acknowledgement

We gratefully acknowledge the Department of Computer Engineering and Technology, Dr. Vishwanath Karad MIT World Peace University, Pune, for providing access to the PARAM Shavak Deep Learning system, equipped with an NVIDIA RTX A4500 GPU. The computational resources made available through this system were crucial for the training of the ThermalODE model across several gigabytes of GPU telemetry data.

We extend sincere thanks to Dr. Pratvina Talele (Project Guide) and Dr. Balaji M Patil (Program Director), Department of Computer Engineering and Technology, MIT World Peace University, for providing mentorship, constructive feedback and institutional support required to make this project possible.

We are also thankful to the MIT Supercloud team for making the TX-Gaia cluster telemetry dataset publicly available through the MIT Data Center Challenge (DCC) repository. The real-world production data from NVIDIA Volta V100 GPUs formed the backbone of this entire project.

Saket Sontakke

Taha Kapadia

Pranjal Vishwakarma

Akshat Srivastava

Abstract

The increase in Artificial Intelligence (AI) and Deep Learning (DL) workloads has resulted in exponential growth in computational demands within High-Performance Computing (HPC) clusters. To satisfy these requirements, modern HPC cluster nodes are composed of dense, multi-GPU accelerated processors. However, this concentration of processors generates a significant amount of heat, elevating node temperatures to levels where the hardware is adversely affected or, in worse cases, enters a throttled state to prevent damage. During this throttled state, clock speeds are reduced, degrading the computational power of the processor and the overall efficiency of the cluster. This paper presents ThermalODE, an Ordinary Differential Equations (ODE) based physics-informed model trained on real-world telemetry from the MIT Supercloud Dataset to simulate the thermal behaviour of individual nodes. The model was evaluated on over 443 million unseen timesteps across 671 jobs, and achieved a global Root Mean Square Error (RMSE) of 2.28°C and Mean Absolute Error (MAE) of 1.45°C, validating its use as a digital twin for the cluster. This digital twin is then deployed within a JIT-compiled cluster simulator to demonstrate its utility as a scheduling policy sandbox. A/B testing of a standard First Come First Serve (FCFS) scheduler against a thermal-aware predictive scheduler was conducted across 921 heterogeneous MIT Supercloud job traces at eight node scales (2 to 256 nodes) under default cooling conditions. Both schedulers successfully completed all 921 jobs with zero throttling events, with makespan scaling from 5,598,808 seconds at 2 nodes to 127,718 seconds at 256 nodes, a 43.8× reduction, validating the simulator's correct modelling of parallel execution and thermal physics under production workload conditions.

Keywords: High-Performance Computing (HPC), Ordinary Differential Equations (ODE), Physics-Informed Machine Learning, Digital Twin, Thermal Management.

List of Figures

List of Tables

Contents

Chapter 1
Introduction

The exponential growth of artificial intelligence workloads, particularly large language models (LLMs) and deep learning frameworks, has increased the computational demand within HPC clusters substantially. To satisfy these requirements, modern architectures rely heavily on dense, multi-GPU accelerated server nodes. This high concentration of processors consumes a large amount of power and as a result considerable heat is produced. Elevated temperatures not only accelerate hardware degradation, but also increase the energy costs associated with datacenter Heating, Ventilation, and Air Conditioning (HVAC) systems. Traditional HPC workload managers like SLURM [1] allocate resources mainly on the availability of the nodes, job priority, and number of requested cores. However, the thermal condition of GPUs to which the tasks are to be assigned is unknown to the scheduler. As a result, when multiple intensive tasks exhaust the GPUs to reach a threshold e.g. 87.0 °C on NVIDIA Volta architectures [2], then the processor's clock speed is reduced, this event is known as throttling. Throttling saves the hardware from being damaged but increases the execution time taken by the task to be completed. GPUs also have a critical shutdown threshold e.g. 90.0 °C on NVIDIA Volta architectures at which point the hardware triggers an emergency shutdown to prevent permanent silicon damage. In order to solve such challenges a scheduler must be able anticipate thermal conditions before jobs are allocated to the GPU, rather than reacting after throttling has already occurred. A model that can accurately predict thermal behaviour of GPUs is crucial. Traditional Computational Fluid Dynamics (CFD) models are too computationally expensive and time consuming for real-time scheduling decisions. Pure data-driven models tend to overfit, they do not adhere to physical constraints and thus can predict values that violate the laws of thermodynamics. Physics-informed ODEs provide a middle ground as they are lightweight enough for real-time use and heat balance equations govern the ODEs which ensures that the predicted results remain within physical bounds. This paper presents ThermalODE, a physics-informed digital twin and proactive thermal-aware scheduler for HPC clusters. The primary contributions of this work are as follows:

1.1 Thermodynamic Prior Extraction:

Empirical calculation of real-world lumped-parameter thermal priors, thermal resistance, convective cooling coefficients, and cross-GPU crosstalk fractions directly from production NVIDIA V100 hardware telemetry of the MIT TX-Gaia cluster, providing reusable physics-grounded constants for future HPC thermal modelling work.

1.2 Physics-Informed ODEs:

A Two-Mass ODE model trained using the PyTorch deep learning library [3] via bounded gradient descent that achieves a global test RMSE of 2.28°C across over 443 million unseen timesteps, validating its deployment as an accurate digital twin of the MIT TX-Gaia cluster.

1.3 Thermal-Aware Scheduling Simulator:

A cluster simulator built using Numba Just-in-time (JIT) compiler [4] on the calibrated ODEs, capable of simulating standard FCFS (thermally blind) scheduler, a proposed thermal aware scheduler or both using A/B testing feature.

1.4 Interactive Web Dashboard:

A visualization web application interface for telemetry replay and scheduling.

Chapter 2
Literature Survey

With increased computing power comes higher thermal density, which is becoming a major challenge for resource managers in High-Performance Computing (HPC). Traditional, "thermally blind" resource managers have become ineffective especially under conditions of long-running intensive jobs. In our analysis of the recent scientific literature, we distinguish three directions where the field has seen progress: reactive scheduling, model-based prediction of temperature, and digital twin-based orchestration.

2.1 Reactive Scheduling and Hardware

Throttling Traditional cluster orchestration is based on resource utilization rates and job priority rules without taking into account the actual thermodynamic state of the compute nodes. Recent empirical studies revealed that thermal hotspots along with Dynamic Voltage and Frequency Scaling (DVFS) were key causes behind execution skews and reduced throughput in multi-GPU systems [5]. Initial research into solving this problem involved only node-level optimization, such as thermal aware scheduling frameworks that demonstrated the ability to cool down processors between 6°C to 12.2°C in single-node integrated CPU-GPU systems [6]. However, implementing similar heuristics on the scale of distributed HPC clusters is highly complicated due to the need for harsh power capping that affects makespan efficiency of the system [7].

2.2 Thermal Prediction Using Data Analysis

In shifting toward proactive control, the HPC domain has extensively focused on using machine learning (ML) to predict thermal conditions. Machine learning (ML) prediction techniques and neural networks have proved their effectiveness in predicting short-term temperature rises [8]. Extending these studies, deep reinforcement learning (RL) methods have been applied to schedule computational resources in clusters to prevent thermal emergencies by placing workloads intelligently and managing cooling systems [9]. Purely data-driven approaches exhibit great precision when dealing with scenarios similar to those used in training but are not physically meaningful and tend to break down in extreme scenarios outside the

training distribution range, which has led to exploration of physically-consistent networks [10]. MIT Supercloud TX-Gaia cluster Partition 1 Partition 2 node 1 node 2 node 224 node 1 node 2 node 480 Intel Xeon Gold 6248 (20 core) NVIDIA Volta V100 Intel Xeon Gold 6248 (20 core) NVIDIA Volta V100 Intel Xeon Platinum 8260 (24 core) Intel Xeon Platinum 8260 (24 core) Fig. 1. Structural hierarchy of the MIT Supercloud TX-Gaia cluster.

2.3 Digital Twins and Predictive Control

In the present state-of-the-art, digital twins are utilized to simulate and assess scheduling schemes before their execution. Large-scale supercomputing centers stressed the importance of using digital twins for the evaluation of the intricate trade-off between job scheduling policies and power and cooling incentives [11]. Environments like DataCenterGym and SustainGym provide macro-scale simulations grounded on physical processes that include HVAC systems and the consideration of carbon footprint. On the other hand, systems like TAPAS [12] and SchedTwin [13] use digital twins to simulate at runtime the scheduling policies and dynamically adjust placement schemes, achieving notable recovery of throughput on inference workloads within warm datacenters.

Chapter 3
Problem Statement

Traditional HPC schedulers such as SLURM allocate compute resources without any knowledge of the thermal state of GPU accelerators. When multiple thermally intensive AI workloads are co-scheduled on the same or adjacent nodes, GPUs can rapidly approach their thermal throttle threshold (87.0°C on NVIDIA Volta architectures), causing automatic clock-speed reductions that degrade computational throughput and extend job completion times. In extreme cases, GPUs may reach the critical shutdown threshold (90.0°C), triggering hardware shutdowns and potentially causing permanent silicon damage.

The core problem is the absence of a computationally lightweight, physically grounded model that can predict GPU thermal behaviour at scheduling time with sufficient accuracy to enable proactive, thermally-aware job placement decisions at the cluster scale.

3.1 Project Scope

Telemetry data was sourced from the MIT Supercloud Dataset [14], which collected telemetry data from the MIT Supercloud TX-Gaia cluster. The TX-Gaia system consists of two primary partitions. The first partition contains 224 nodes, each equipped with two 20-core Intel Xeon Gold 6248 processors (384GB RAM) and two NVIDIA Volta V100 GPUs (32GB RAM each). The second partition is entirely CPU based, comprising 480 nodes with two 24-core Intel Xeon Platinum 8260 processors (192GB RAM) per node. The scope explicitly excludes CPU thermal modelling, HVAC and facility-level cooling dynamics, liquid cooling systems, and GPU architectures other than NVIDIA Volta V100.

As illustrated in Figure. 3.1, this study is scoped exclusively to the NVIDIA Volta V100 GPUs within Partition 1 (green). Partition 2 was excluded entirely as it contains no GPU accelerators and is therefore irrelevant to GPU thermal modelling. Within Partition 1, the host Intel Xeon CPUs were also excluded for two reasons: fine-grained CPU temperature telemetry was not available in the dataset, and CPUs serve primarily as job dispatchers in GPU-accelerated HPC

workloads rather than as the primary computational workhorses. The V100 GPUs are where the overwhelming majority of compute, and by extension heat, is generated during AI and DL jobs, making them the sole relevant subject of thermal analysis.

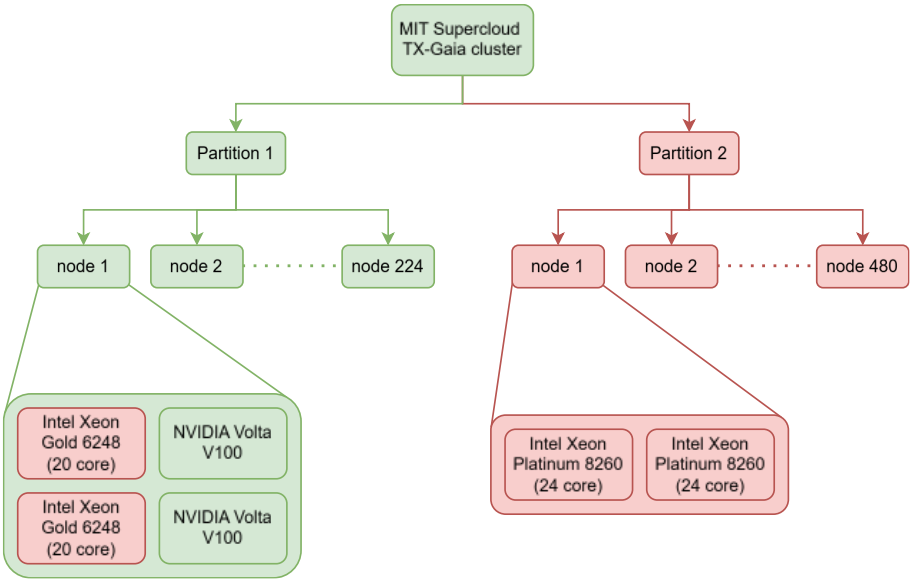

Figure 3.1: Structural hierarchy of the MIT Supercloud TX-Gaia cluster

The project encompasses the full pipeline from raw telemetry data to a deployable scheduling sandbox, specifically:

- Data preprocessing pipeline for MIT Supercloud GPU telemetry (temperature, power draw, GPU and memory utilization at 100ms intervals).
- Empirical extraction of thermodynamic priors (thermal capacitance, resistance, convective cooling, crosstalk coefficients) from production V100 hardware behaviour.
- Design, training, and validation of a Two-Mass physics-informed ODE model for dual-GPU node thermal dynamics.
- Implementation of a JIT-compiled cluster simulator with FCFS and Thermal-Aware schedulers and A/B testing capability.
- Development of a browser-native interactive dashboard for telemetry replay and scheduling policy visualization.

3.2 Project Assumptions

- GPU thermal behaviour follows a lumped-capacitance Two-Mass model (silicon die + heatsink) with dynamic sigmoid-gated convective cooling, which is an accepted engineering approximation for discrete electronic thermal assemblies.
- Ambient datacenter temperature is approximately constant at 30°C for the A/B testing experiments (controlled simulation parameter).
- The MIT Supercloud Dataset provides representative samples of production HPC workload diversity, and the 20% data subset (20 of 100 subfolders) used for training is sufficient for generalization.
- The Forward Euler integration at $\Delta t = 0.11$ s provides sufficient temporal resolution given the thermal time constants of V100 GPU heatsinks ($\tau \approx$ hundreds of seconds).
- Power draw traces from historical job logs are valid proxies for the power draw that would occur if those jobs were re-run in the simulator.

3.3 Project Limitations

- The current model is calibrated for NVIDIA Volta V100 GPUs specifically. Application to other GPU architectures would require re-calibration of all 18 trainable parameters using telemetry from the target hardware.
- The simulator fixes ambient temperature as a constant; real datacenters exhibit dynamic ambient conditions that respond to cluster thermal load through HVAC feedback.
- CPU thermal dynamics are not modelled, as fine-grained CPU temperature telemetry was not available in the MIT Supercloud Dataset.
- The maximum observed absolute prediction error of 34.68°C occurs during rapid power-step transients, a known limitation of explicit first-order integration under steep gradient conditions.
- The thermal-aware scheduler does not outperform FCFS under nominal cooling conditions, as the thermal advantage of proactive placement only manifests when nodes operate near thermal saturation.

3.4 Project Objectives

- O1: Data preprocessing and extraction of empirical thermodynamic priors (C, R_{th}, h, κ) from production NVIDIA V100 telemetry using crosstalk event analysis and step-response curve fitting.
- O2: Design and implement a Two-Mass physics-informed ODE model with dynamic sigmoid-gated cooling that respects thermodynamic constraints through bounded parameter optimization.
- O3: Achieve a respectable global test-set RMSE across the unseen test dataset, validating deployment as a digital twin.
- O4: Implement a JIT-compiled cluster simulator with A/B testing of FCFS and Thermal-Aware schedulers across configurable node counts and cooling scenarios.
- O5: Develop and deploy a browser-native interactive dashboard for real-time scheduling policy visualization without requiring a local Python environment.

Chapter 4

Project Requirements

The project was developed by a team of four undergraduate researchers from the Department of Computer Science and Engineering, Dr. Vishwanath Karad MIT World Peace University, Pune.

3.5 Hardware Requirements

3.5.1 Training GPU:

- Specification: NVIDIA RTX A4500 (16 GB VRAM)
- Purpose: ODE model training; VRAM holds the full 39,533-segment validation bundle permanently resident during training.

3.5.2 CPU & RAM:

- Specification: Multi-core x86-64; ≥ 32 GB RAM recommended
- Purpose: Hosts the *MultiChunkPrefetcher* daemon; RAM caches training chunks between the SSD and GPU VRAM via pinned memory buffers.

3.5.3 Storage:

- Specification: SSD; ≥ 500 GB available space
- Purpose: Stores raw Parquet files, chunked .pt tensor files (4 training chunks), 671 individual test .pt files, and simulation output ZIP archives.

3.6 Software Requirements

3.6.1 Python:

- Version: ≥ 3.9 (Runtime)
- Usage: Core language for preprocessing, prior extraction, and model training.

3.6.2 PyTorch:

- Version: ≥ 2.0 (ML Framework)

- Usage: ODE training, Adam optimizer, *ReduceLROnPlateau* scheduler, masked MSE loss.

3.6.3 Numba:

- Version: ≥ 0.57 (JIT Compiler)
- Usage: @njit(cache=True) compilation of *step_physics_numba* and *find_best_placement_numba*.

3.6.4 NumPy:

- Version: ≥ 1.24 (Numerical)
- Usage: Array operations, memory-mapped I/O for large trace files, .npy binary cache.

3.6.5 pandas / PyArrow:

- Version: Latest stable (Data Processing)
- Usage: CSV-to-Parquet conversion, *merge_asof* GPU stream alignment, Parquet scanning.

3.6.6 SciPy:

- Version: ≥ 1.10 (Scientific)
- Usage: *scipy.optimize.curve_fit* for exponential step-response heating curve extraction.

3.6.7 CUDA Toolkit:

- Version: ≥ 11.8 (GPU Runtime)
- Usage: Non-blocking CUDA streams for VRAM prefetch; CUDA-aware tensor operations.

3.6.8 TypeScript / Node.js:

- Version: ≥ 5.0 / ≥ 18 (Web Frontend)
- Usage: Browser physics port, Web Worker simulation engine, delta compression, OPFS export.

3.6.9 React / Next.js:

- Version: ≥ 18 / ≥ 14 (Web Framework)

- Usage: Interactive dashboard UI hosted at https://thermal-ode.vercel.app/.

3.6.10 Chart.js:

- Version: ≥ 4.0 (Visualization)
- Usage: Real-time telemetry charts in the web dashboard; embedded in HTML export files with pan-and-zoom.

3.6.11 PapaParse:

- Version: ≥ 5.0 (CSV Parsing)
- Usage: Browser-native CSV ingestion with the worker:true flag to offload parsing off the main UI thread.

3.6.12 Vercel:

- Version: N/A (Deployment)
- Usage: Hosting and CDN for the interactive web dashboard; no local HPC infrastructure required for end users.

Chapter 5

System Analysis and Proposed Architecture

The architecture of ThermalODE was shaped by three overarching design imperatives. First, physical plausibility: all model predictions must remain within thermodynamically feasible bounds at all times, even under extrapolation to conditions not encountered during training. Second, computational efficiency: the thermal model must be lightweight enough for real-time forward projection during scheduling decisions, which precludes computationally expensive approaches such as CFD simulation. Third, empirical grounding: model parameters must be initialized from and constrained by values derived from real hardware telemetry rather than generic engineering tables, ensuring that the digital twin accurately represents the specific physical characteristics of the MIT TX-Gaia cluster.

5.1 Design Considerations

These design imperatives directly motivated the Three iterative architectural decisions documented in Chapter 7: Single-Mass Static Cooling (insufficient for non-linear fan dynamics), Single-Mass Dynamic Cooling (insufficient for capturing die-heatsink thermal decoupling), and the final Two-Mass Dynamic Cooling architecture adopted in the production system.

5.2 System Architecture

The ThermalODE system comprises five major architectural components that operate in a pipeline from raw data to interactive visualization:

5.2.1 Data Preprocessing Pipeline:

Converts raw MIT Supercloud GPU telemetry (CSV) to Parquet format, applies dual-GPU alignment with nearest-neighbour merge, stratifies job traces across length and thermodynamic density dimensions, and generates fixed-length PyTorch tensor segments for training.

5.2.2 Thermodynamic Prior Extractor:

Scans the dataset for asymmetric thermal events and step-response heating curves to

empirically derive thermal capacitance (C), resistance (R_{th}), convective cooling coefficients (h), and inter-GPU crosstalk fractions (κ) for both GPU 0 and GPU 1.

5.2.3 ThermalODE Model:

A Two-Mass ODE system with 18 trainable parameters, trained via PyTorch with the Adam optimizer and enforced physical bounds through sigmoid projection. Uses Forward Euler integration at $\Delta t = 0.11$ s against 100ms-sampled real telemetry.

5.2.4 JIT-Compiled Cluster Simulator:

A Numba-accelerated simulation engine that embeds the calibrated ODE, enforces hardware throttle/recovery/shutdown logic, and implements FCFS and Thermal-Aware scheduling with A/B testing capability across configurable node counts.

5.2.5 Web Dashboard:

A browser-native TypeScript application that ports the complete simulation stack to the browser runtime, providing real-time visualization through Web Workers, Chart.js rendering, and OPFS-backed high-resolution data export.

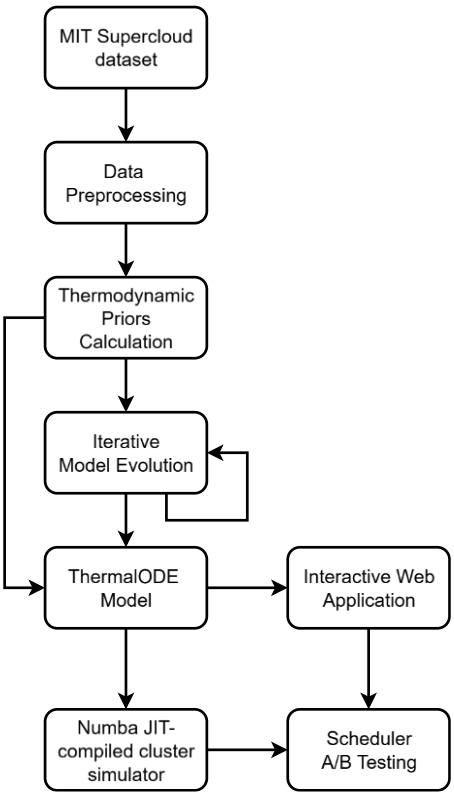

Figure 5.1: System Architecture Diagram

5.3 Modules of the Project

5.3.1 Module 1: Data Preprocessing

Responsible for acquiring the MIT Supercloud Dataset, converting CSV files to Parquet format, filtering dual-GPU job files, applying nearest-neighbour temporal alignment, computing per-job ambient temperature estimates, and generating stratified tensor datasets. Implements the 2D stratification matrix (9 strata × 3 splits) yielding 5,299 training, 659 validation, and 671 test jobs.

5.3.2 Module 2: Thermodynamic Prior Extraction

Responsible for identifying asymmetric thermal events (one GPU active ≥ 150W, adjacent GPU idle ≤50W) to compute directional crosstalk coefficients k_{01} and k_{10}. Also implements step-response exponential curve fitting using *scipy.optimize.curve_fit* with bounded constraints to derive per-GPU values of C, R_{th}, h, and the dimensionless crosstalk prior κ. Processes 6,629 dual-GPU job files, yielding valid priors from 2,453 files.

5.3.3 Module 3: ThermalODE Model Training

Responsible for implementing the Two-Mass ODE architecture in PyTorch, including the sigmoid-gated dynamic cooling function $h(T_{die})$, the Forward Euler integration loop, the sigmoid parameter bounding mechanism, the masked MSE loss function, the *ReduceLROnPlateau* learning rate scheduler, and the *MultiChunkPrefetcher* for efficient multi-gigabyte dataset training on the NVIDIA RTX A4500. Manages 18 trainable parameters with empirically initialized priors and physical bounds.

5.3.4 Module 4: Cluster Simulator and Web Dashboard

Responsible for implementing the Numba JIT-compiled cluster simulator, including: job ingestion with Welford online statistics, hardware throttle/recovery/shutdown state machine, FCFS Standard scheduler, Thermal-Aware predictive scheduler with 2,727-timestep forward projection, and A/B testing framework. Also responsible for the TypeScript port of the physics engine, Web Worker simulation architecture, master-clock A/B lockstep synchronization, Chart.js real-time visualization, delta-compression state transmission, and OPFS-backed high-resolution export.

5.4 UML Diagrams

5.4.1 Use Case Diagram

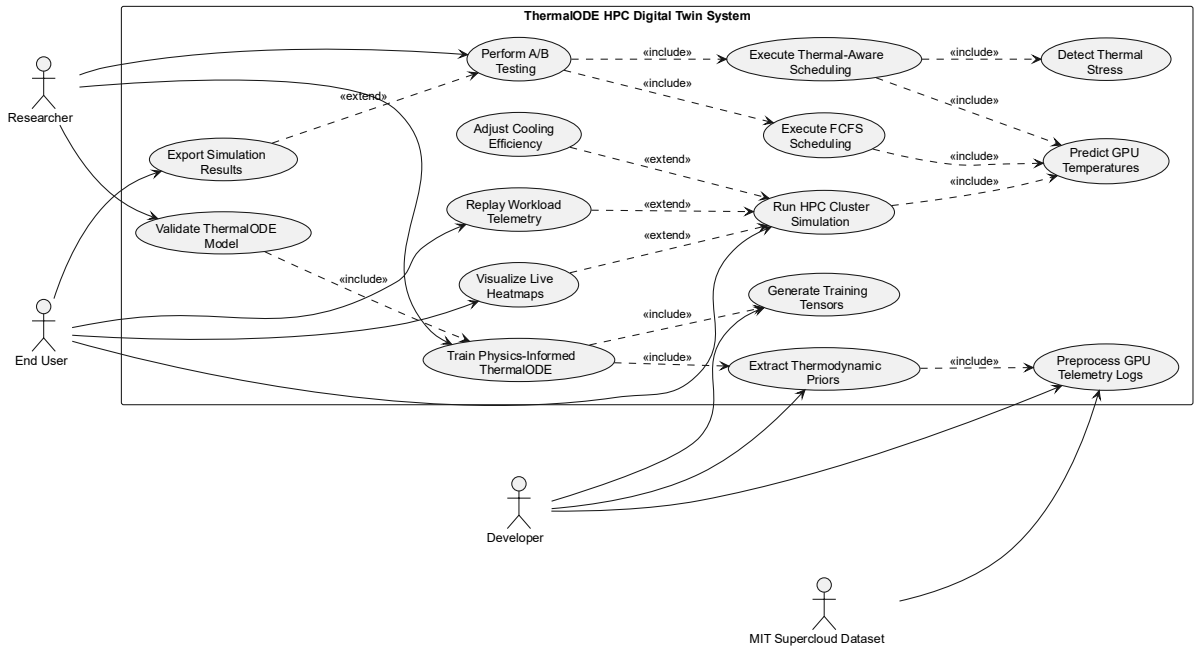

Figure 5.2: Use Case Diagram

5.4.2 Class Diagram

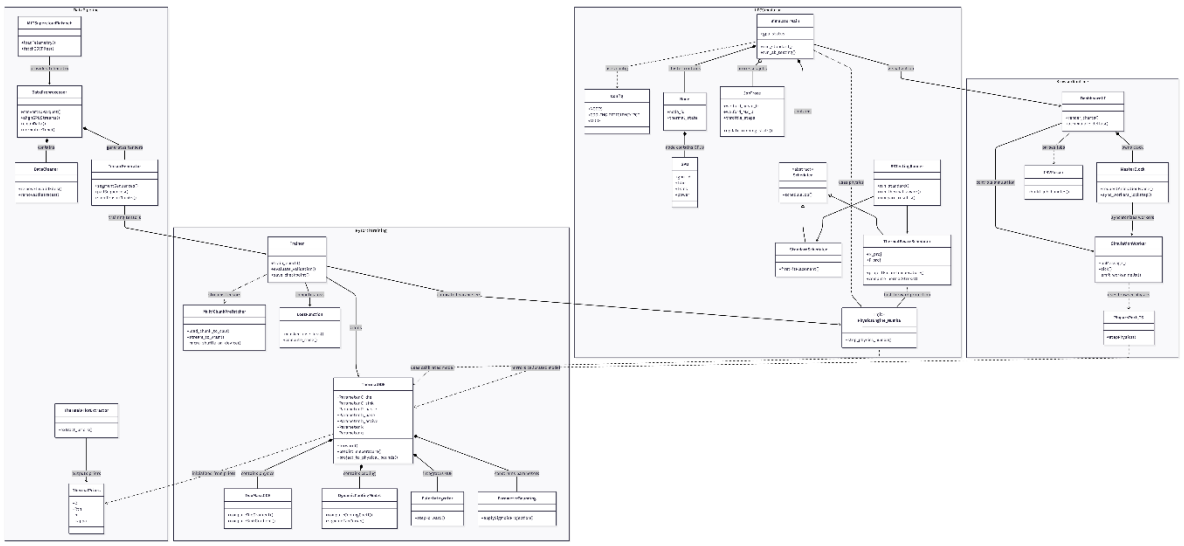

Figure 5.3: Class Diagram

5.4.3 Activity Diagram

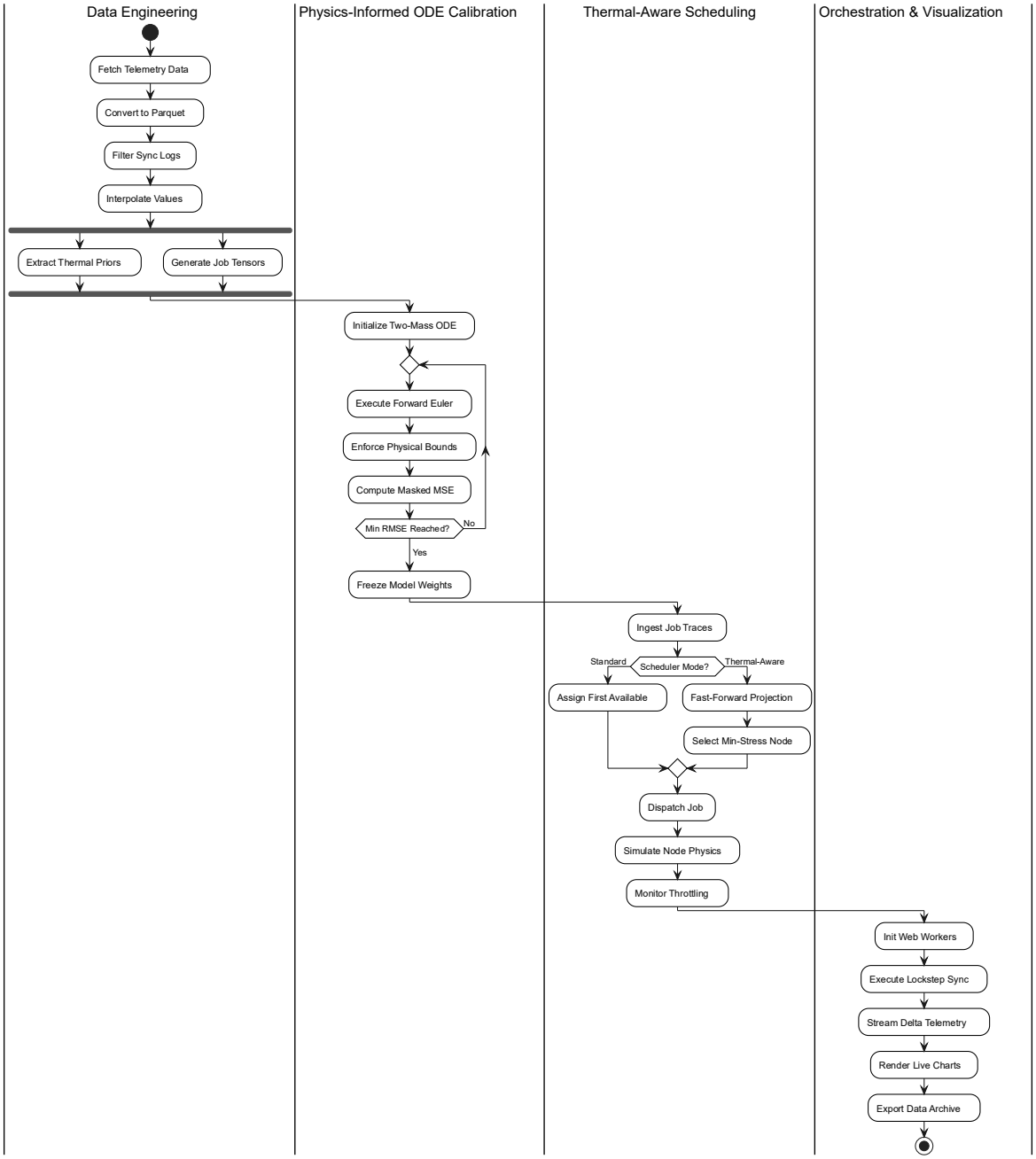

Figure 5.4: Activity Diagram

Chapter 6
Project Plan

ThermalODE was developed using an iterative, data-driven methodology closely aligned with the Agile framework. The project proceeded through six major phases, with each phase producing validated deliverables before the next began. The model architecture specifically evolved through three explicitly documented iterations (Single-Mass Static, Single-Mass Dynamic, Two-Mass Dynamic), with each iteration informed by training log analysis and residual error investigation rather than a pre-planned design. This approach is described in detail in Chapter 7.

6.1 Project Timeline

The following table outlines the major milestones and their approximate durations across the project lifecycle:

Phase	Milestone	Key Activities Natural Language	Duration
Phase 1	Dataset Acquisition & Preprocessing	MIT Supercloud dataset download, CSV-to-Parquet conversion, dual-GPU alignment, stratified splitting, tensor generation	3 weeks
Phase 2	Thermodynamic Prior Extraction	Asymmetric event detection, crosstalk coefficient calculation, step-response curve fitting, prior validation	2 weeks
Phase 3	Model Architecture Design	Iteration 1 (Single-Mass Static) – training and residual analysis	1 week
Phase 3	Model Architecture Design	Iteration 2 (Single-Mass Dynamic) – training and parameter saturation analysis	2 weeks
Phase 3	Model Architecture Design	Iteration 3 (Two-Mass Dynamic) – full training to convergence at epoch 81	1 week

Phase 4	Model Evaluation	Global RMSE/MAE, hexbin density analysis, tail-risk evaluation (danger zone), temporal stability analysis	1 week
Phase 5	Cluster Simulator Development	JIT compilation, scheduler logic, A/B testing framework, output artifacts, 8-node-scale experiment	3 weeks
Phase 6	Web Dashboard Development	TypeScript physics port, Web Worker architecture, Chart.js rendering, A/B lockstep, export pipeline, Vercel deployment	3 weeks

Table 6.1: Project Timeline

Chapter 7
Implementation

The implementation methodology combines three established engineering disciplines: (i) empirical thermodynamic system identification, (ii) physics-informed machine learning, and (iii) high-performance systems programming. The central methodological contribution is the use of empirically extracted thermodynamic priors to initialize and bound an ODE-based learnable model, which is then trained end-to-end against real telemetry. This hybrid approach avoids the limitations of both pure CFD (too slow for real-time use) and pure ML (physically unconstrained).

7.1 Data Acquisition and Preprocessing

7.1.1 Dataset Acquisition

The MIT Supercloud dataset provides a combination of high-level workload manager data and low-level job-specific Comma-Separated Values (CSV) time series data. GPU telemetry data, collected via the *nvidia-smi* utility at 100ms intervals, constitutes terabytes of time-series logs. To optimize development and computational overhead while maintaining a massive, representative sample, the preprocessing pipeline was initialized using 20% of the total available GPU subfolders (20 out of a total of 100 subfolders, specifically *gpu/0000* through *gpu/0019*). The dataset is publicly available and was acquired through the MIT Data Center Challenge repository[1].

7.1.2 Data Filtering and Cleaning

Acquired raw GPU time-series data consisted thousands of CSV files adding up to several hundred gigabytes in size. To reduce storage overhead and speed up downstream processing, all CSV files were first converted to Apache Parquet format using PyArrow. The resulting Parquet files were then scanned to identify nodes with consistent dual-GPU telemetry. A file was retained only if logs for both GPU 0 and GPU 1 were present, their row counts differed by no more than 5%, and their timestamp boundaries were within 5 seconds of each other. Files

[1] https://dcc.mit.edu/data/

failing either condition were discarded, as misaligned GPU streams cannot be reliably paired for two mass ODE training.

Filtered files were then aligned and cleaned. The asynchronous GPU 0 and GPU 1 telemetry streams were merged using a nearest-neighbour *merge_asof* with a 0.05-second tolerance window, and the unified timestamp was set to the midpoint of each matched pair. Absolute timestamps were then normalized to elapsed time from t = 0. Three additional filters were applied: files missing required columns (temperature, power draw, GPU and memory utilization) were dropped; files with fewer than 50 paired rows were discarded as too short for meaningful ODE calibration; and files with no thermodynamic excitation (defined as no single timestep with a temperature delta ≥ 1.0 °C or power delta ≥ 5.0W across either GPU) were removed, as static idle traces carry no useful thermal dynamics for training.

7.1.3 Stratified Splitting and Tensor Generation

To ensure that ThermalODE generalized across a diverse range of operational conditions, a global 2D stratification strategy was employed prior to the dataset split. Table 7.1 illustrates the binning across two dimensions: temporal length (Short, Medium, Long) and thermodynamic density (Low, Medium, High).

Length / Density	**Low**	**Medium**	**High**	**Total**
Short	938	1,028	244	2,210
Medium	401	715	1,093	2,209
Long	871	466	873	2,210
Total	2,210	2,209	2,210	**6,629**

Table 7.1: 2D Stratification Matrix of Physical Job Trajectories

A deterministic 80%, 10% and 10% split was then applied within each stratum which resulted in 5,299 jobs in training set, 659 jobs in validation set of, and 671 jobs in test set, as detailed in Table 7.2. Furthermore, the ambient temperature T_{amb} was estimated per job, as it is required

by the ODE as an initial boundary condition but is not directly recorded in the available timeseries data.

Strata	Total	Train (80%)	Val (10%)	Test (10%)
Short Low	938	750	93	95
Short Medium	1028	822	102	104
Short High	244	195	24	25
Medium Low	401	320	40	41
Medium Medium	715	572	71	72
Medium High	1093	874	109	110
Long Low	871	696	87	88
Long Medium	466	372	46	48
Long High	873	698	87	88
Grand Total	**6629**	**5299**	**659**	**671**

Table 7.2: Detailed 80-10-10 Splitting for Strata Trajectories

The first 100 rows of each job trace were scanned for an idle period where both GPU utilizations were zero. If found, the cooler of the two GPUs at that timestep was used to back-calculate T_{amb} by inverting the steady-state heat transfer equation:

$$T_{\text{amb}} = T_{\text{die}} - \frac{P_{\text{self}}}{h} \quad (1)$$

where h is the empirically derived convective cooling prior for that GPU. If no idle period existed (a hot-start job), the first row was used as the fallback. This approach avoids the need for a dedicated ambient sensor in the telemetry stream. The resulting split datasets consisted of 1,705,242,321, 196,157,875, and 221,907,690 timesteps in training, validation and testing set

respectively. These were then combined into fixed-length segments of 5,000 timesteps each which resulted in 343,503 training segments, 39,533 validation segments, and 44,691 test segments. Shorter terminal segments were zero padded to maintain uniform tensor dimensions. A branching stack strategy was implemented to prevent RAM exhaustion during training:

- Training Data: 100,000 segments were stacked into a single PyTorch .pt chunk file, producing a total of 4 chunk files across the 343,503 training segments.
- Validation Data: All 39,533 segments were packed into a single tensor file for high-speed evaluation after every epoch.
- Test Data: Preserved with a strict 1-to-1 physical mapping (one .pt file per original Parquet job) across 671 test jobs, guaranteeing evaluation over contiguous, uncorrupted physical trajectories.

7.2 Thermodynamic Priors Extraction

A physics-informed ODE cannot be trained the same way as a conventional black-box neural network. Rather than allowing gradient descent to search an unbounded parameter space, the physical constants governing heat transfer such as thermal capacitance, resistance, and cooling coefficients must be constrained to values that are thermodynamically feasible for the specific hardware being modelled. These constraints are referred to as thermodynamic priors. These priors are empirically derived estimates of the true physical constants of the NVIDIA V100 GPUs in the MIT TX-Gaia cluster, calculated directly from the telemetry data before any model training begins. Providing these priors as accurately as possible achieves two purposes. Firstly, they initialize the ODE parameters close to their true physical values, which gives the gradient descent a reasonable starting point rather than a random arbitrary one. Secondly, lower and upper bounds can be applied on these parameters, preventing the model from learning physically impossible behaviour such as negative thermal mass or heat flowing from cold to hot. The cleaned dataset consisted of 6,629 dual-GPU job files and these were analyzed in two stages to extract the priors:

- Calculation of thermal crosstalk coefficients (k), the heat transfer from one GPU to its neighbouring GPU residing in the same node.
- Calculation of lumped-parameter physical constants, namely thermal capacitance (C), thermal resistance (R_{th}), convective cooling coefficient (h), and the dimensionless crosstalk prior (κ).

7.2.1 Thermal Crosstalk Calculation

In cluster nodes containing dual-GPU hardware, adjacent GPUs share a local thermal environment, resulting in passive heat transfer between the GPUs residing within the same node. To accurately capture these node level thermal dynamics the dataset needs to be standardized. The dual-GPU data that was obtained after data cleaning was re-sampled into uniform intervals of one second. Missing values were filled in with estimates using linear interpolation, ensuring a continuous and synchronized time-series for thermal crosstalk calculation. To isolate pure thermal crosstalk and eliminate multicollinear noise from concurrent workloads, the dataset was strictly scanned to locate Asymmetric Events. An asymmetric event was defined as a continuous block of at least 30 seconds in which one GPU was subjected to a sustained heavy workload (power draw $P_{active} \geq 150.0$ W) while the adjacent GPU remained entirely idle (power draw $P_{idle} \leq 50.0$ W). An event was retained only if the idle GPU exhibited a passive temperature rise of $\Delta T \geq 1.0$ °C above its baseline temperature at event onset. For each valid event, the directional thermal crosstalk coefficient was computed as:

$$k = \frac{\Delta T}{\overline{P_{\text{active}}}} \tag{2}$$

where $\overline{P_{\text{active}}}$ is the mean power draw of the active GPU over the event window. Across 6,628 successfully processed files, 1,270 asymmetric events isolating GPU 1 heating GPU 0 were identified, yielding a mean crosstalk coefficient $k_{01} = 0.01602$ °C/W. A total of 93 events isolating GPU 0 heating GPU 1 were identified, yielding $k_{10} = 0.00522$ °C/W.

7.2.2 Lumped Parameter Thermal Prior Extraction

Following the establishment of directional crosstalk coefficients, the intrinsic thermodynamic parameters of each GPU were extracted. The cleaned dataset of 6,629 dual-GPU job files was scanned for step-response heating curves, defined as intervals initiated by a rapid power spike exceeding 100.0 W, sustained for at least 30 seconds, and captured within a maximum observation window of 150 seconds. Files where power dropped below 50% of the spike threshold before 30 seconds elapsed were discarded as insufficiently sustained. For each identified heating phase, the telemetry was first resampled to uniform 1-second intervals via linear interpolation. An exponential step-response curve of the form:

$$T(t) = T_{\text{idle}} + T_{\text{amp}}\left(1 - e^{-t/\tau}\right) \tag{3}$$

was fitted to the heating trace using *scipy.optimize.curve_fit* with bounded constraints: $T_{amp} \in [3.0, 100.0]$ °C and $\tau \in [5.0, 400.0]$ s. Curves where the fitted τ collided with either boundary were rejected as physically implausible. From the accepted fits, the lumped thermal resistance and convective cooling coefficient were derived as:

$$R_{th} = \frac{T_{\text{amp}}}{P_{\text{heat}} - P_{\text{idle}}} \tag{4}$$

$$h = \frac{1}{R_{th}} \tag{5}$$

$$C = \frac{\tau}{R_{th}} \tag{6}$$

with a final sanity check requiring $C \in [20.0, 1000.0]$ J/°C. The previously calculated thermal crosstalk coefficients (k_{01} = 0.01602 °C/W and k_{10} = 0.00522 °C/W) are converted to dimensionless crosstalk priors using:

$$\kappa_{ij} = k_{ij} \cdot h_i \tag{7}$$

Of the 6,629 scanned files, 2,453 yielded valid step-response curves, 1,624 contributing fits for GPU 0 and 2,148 for GPU 1. The remaining 4,176 files were discarded due to insufficient thermal excitation or curve-fit rejection. The median priors extracted across valid fits are summarized in Table 7.3.

Parameter	**Unit**	**GPU 0**	**GPU 1**
Capacitance Prior (C)	J/°C	143.22	140.25
Resistance Prior (R_{th})	°C/W	0.2053	0.1856
Convective Cooling Prior (h)	W/°C	4.8713	5.3871

Crosstalk Prior (κ)	Dimensionless	0.0780	0.0281

Table 7.3: Thermal Priors for GPU 0 and GPU 1

The close agreement between C_0 and C_1 (less than 2.1% relative difference) supports the assumption of similar thermal mass across the two GPUs, consistent with their identical hardware specification. From the observed priors and the divergence in h values it can be inferred that airflow conditions within the node are asymmetric. GPU 1 likely sits upstream in the airflow path, meaning its thermal exhaust is carried directly over GPU 0, which explains the higher crosstalk coefficient (k_{01} = 0.01602 °C/W) and higher thermal resistance ($R_{th,0}$ = 0.2053 °C/W) observed for GPU 0. Conversely, heat generated by GPU 0 must propagate against the primary airflow to reach GPU 1, resulting in a significantly lower crosstalk coefficient (k_{10} = 0.00522 °C/W) and lower thermal resistance ($R_{th,1}$ = 0.1856 °C/W) for the upstream GPU.

7.3 Model Architecture Evolution

The final physics-informed ODE architecture evolved through an iterative, data-driven process rather than starting from a single, pre-planned design. The development of the three model iterations was guided by actively monitoring the training logs, with execution halted whenever a model hit mathematical or physical limits. As a result, the first two iterations were manually stopped before reaching their planned epoch budgets. Their performance metrics reflect mid-development snapshots rather than fully converged states; only Iteration 3 was trained to complete convergence. Because of this difference in training budgets, a direct numerical comparison of their final performance is not meaningful and is therefore omitted. To manage the expanding complexity across these iterations, we used the empirically derived values from Section IV as starting points where applicable. However, as the architecture evolved to capture non-linear and multi-mass dynamics, our reliance on these simple step-response priors shifted. Newly introduced parameters were initialized using standard engineering heuristics. Throughout all iterations, the parameters were mathematically restricted to remain within realistic physical limits during training, ensuring the optimizer could not learn impossible behaviours like negative thermal mass. The exact parameter initializations and boundary limits for the final architecture are provided in Table 7.4.

7.3.1 Iteration 1: Single-Mass Static Cooling

The first iteration modelled the entire GPU thermal assembly as a single lumped capacitance C with a static linear cooling coefficient h. For this iteration, the empirically derived priors (C, h, and κ) were used directly as the initial weights. Training was manually stopped at epoch 35 out of a planned 50 after the logs showed signs of early stagnation. The validation RMSE plateaued around 6.16 °C, with improvements dropping below 0.002 °C per epoch. Since the optimizer was stuck in a local minimum and a high error, the training was paused to investigate the bottleneck. Analyzing the residuals at the epoch 35 checkpoint showed that the model consistently underpredicted temperatures above 70 °C, completely failing to track high-temperature plateaus. This made physical sense: a static h parameter represents a constant cooling rate, meaning it mathematically cannot capture the aggressive, non-linear way GPU fan curves engage at higher temperatures. This limitation prompted the move to Iteration 2.

7.3.2 Iteration 2: Single-Mass Dynamic Cooling

To address the fan curve issue, a sigmoid-gated, temperature dependent cooling function h(T) was introduced. While the empirical priors for capacitance (C) and crosstalk (κ) were retained, the static h was replaced by new dynamic cooling parameters (h_{base} and h_{active}), which were initialized using engineering heuristics. This second iteration was set to run for 500 epochs but was manually halted early at epoch 195. Although the validation RMSE initially improved, the error trajectory eventually plateaued with an improvement of only 0.02°C over the last 50 epochs. More importantly, the parameter evolution showed the lumped capacitance C drifting continuously upward until it hit 993.81 J/°C, saturating against our predefined physical upper bound of 1000.0 J/°C. Essentially, the optimizer was trying to inflate the thermal mass to artificially dampen errors it could not physically fit. Due to the error plateau and parameter saturation, the training was stopped. Evaluating the epoch 195 checkpoint revealed persistent tracking errors during sudden power spikes. This led to an understanding that governing the temperature differential dT/dt with a single lumped capacitance C simply cannot capture both the near-instantaneous heating of the die and the slow thermal soaking of the massive heatsink. This proved that the model needed to be structurally decomposed.

7.3.3 Iteration 3: Two-Mass Dynamic Cooling

Guided by previous failure modes, the lumped thermal mass was split into two distinct masses: a silicon die (C_{die}) and a heatsink (C_{sink}), coupled by thermal paste resistance (R_{paste}). Because

the single empirical C value could not accurately represent this decoupled system, it was discarded and heuristic starting points were used for both masses. Ultimately, only the empirical crosstalk prior (κ) was carried forward directly from Section IV. This model was trained to convergence and the parameters quickly stabilized within realistic physical boundaries. Early stopping identified the optimal weights at epoch 81, achieving a minimum validation RMSE of 2.28 °C. It successfully captured both the high-temperature plateaus and the fast power spikes. This two-mass architecture with dynamic cooling was finally adopted for all subsequent analysis.

7.4 The Physics-Informed ODE Model

Following the iterative development process detailed in Section V, the Two-Mass Dynamic Cooling architecture was selected as the optimal framework for representing the thermal behaviour of the dual-GPU nodes. This section provides the comprehensive mathematical formulation of this finalized model.

7.4.1 Two-Mass ODE Architecture

The physical hardware was developed as a Two-Mass model for each GPU, splitting the thermal assembly into two masses: the Silicon Die (C_{die}), where heat is generated, and the Heatsink mass (C_{sink}), where heat is dissipated. Heat flows from the die to the heatsink through the thermal interface material, represented by thermal paste resistance (R_{paste}). The heatsink subsequently dissipates energy into the ambient environment (T_{amb}) via convective cooling (h). Since T_{amb} is not directly measured in the telemetry, it is estimated per job during preprocessing as described in Section III. The heatsink temperature at t = 0 is not directly observable in the available timeseries logs. It is therefore initialized at the start of each sequence by assuming initial thermal equilibrium through the paste:

$$T_{\text{sink}}(0) = T_{\text{die}}(0) - P(0) \cdot R_{\text{paste}} \tag{8}$$

For a given GPU, the continuous-time thermal dynamics are governed by the following system of differential equations:

$$\frac{dT_{\text{die}}}{dt} = \frac{1}{C_{\text{die}}}\left(P - \frac{T_{\text{die}} - T_{\text{sink}}}{R_{\text{paste}}}\right) \tag{9}$$

$$\frac{dT_{sink}}{dt} = \frac{1}{C_{sink}}\left(\frac{T_{die} - T_{sink}}{R_{paste}} + k \cdot P_{adj} - h \cdot (T_{sink} - T_{amb}) + q\right) \quad (10)$$

Equation (9) describes the heat balance directly at the silicon level. As the GPU processes workloads, the die generates heat proportional to its power draw (P_{self}). Because the die possesses very little physical mass, its thermal capacitance (C_{die}) is extremely small, meaning its temperature can spike in a matter of seconds. The only way this heat can escape is through conduction across the thermal paste into the heatsink. This transfer rate is defined by the temperature gradient between the die and the sink, but slowed down by the paste's thermal resistance (R_{paste}). Equation (10) characterizes the behaviour of the heatsink. Unlike the die, the heatsink is massive, and its large thermal capacitance (C_{sink}) acts as a thermal reservoir that heats up significantly slower than the silicon die. It not only accumulates heat flowing in from the die, but also absorbs thermal crosstalk ($k \cdot P_{adj}$) radiating from the adjacent GPU's power draw. To cool down, the heatsink dumps energy into the server's passing airflow. This heat convectional dissipation is driven by the difference between the sink temperature and the ambient intake air (T_{amb}), and is scaled by the dynamic fan cooling rate $h(T_{die})$. Finally, a static bias term (q) is added to compensate for localized background heat from unmodelled components, such as nearby CPUs or neighbouring nodes. The physical grounding of both GPUs within a node is identical but the training process maintains fully independent parameter sets for GPU 0 and GPU 1. As noted in the Section IV, upstream and downstream GPUs experience vastly different airflows and calibrating the parameters for both GPUs separately allows the model to inherently obey these physical asymmetries. A complete breakdown of all the 18 trainable parameters, including their description, physical units, the parameter boundaries and the final optimized values are provided in Table 7.4. Although equations (9) and (10) define the continuous thermal physics of the GPUs, the actual MIT Supercloud telemetry is discrete, sampled approximately every 100 milliseconds. To map these continuous equations into deep learning frameworks such as PyTorch and evaluate them against the data, the Forward Euler method is used to numerically integrate the ODEs. A fixed integration timestep of $\Delta t = 0.11$ seconds is used, which closely aligns with the 100ms logging interval while also accounting for minor sensor latencies. Instead of attempting to solve for a continuous curve, the model steps through the sequence chronologically. At each timestep, it evaluates the current temperature gradients and uses them to project the die and heatsink temperatures forward into the next discrete state:

$$T_{\text{die}}(t + \Delta t) = T_{\text{die}}(t) + \Delta t \cdot \frac{dT_{\text{die}}}{dt} \tag{11}$$

$$T_{\text{sink}}(t + \Delta t) = T_{\text{sink}}(t) + \Delta t \cdot \frac{dT_{\text{sink}}}{dt} \tag{12}$$

By propagating these iterative updates across the entire sequence length of the input tensors, the model generates a complete thermal trajectory. However, computing the heatsink gradient at any given timestep requires knowing the exact convective cooling rate at that precise moment. This requires a mathematical representation of the hardware's variable fan speeds, detailed below.

7.4.2 Dynamic Active Cooling modelling

Datacenter GPUs utilize dynamic fan curves that nonlinearly increase the convective cooling rate when die temperatures exceed a hardware defined threshold. To capture this behaviour while maintaining differentiability for backpropagation, the cooling coefficient h is modeled as a sigmoid-gated function of the current die temperature:

$$h(T_{\text{die}}) = h_{\text{base}} + h_{\text{active}} \cdot \sigma\big(\beta(T_{\text{die}} - T_{\text{thresh}})\big) \tag{13}$$

where h_{base} represents passive baseline airflow at idle fan speeds, h_{active} represents the maximum additional cooling capacity at full fan duty cycle, T_{thresh} is the die temperature at which the BIOS begins spinning up the fans, and β controls the steepness of the fan engagement curve. The sigmoid operator σ is a fixed mathematical function and carries no calibrated value of its own.

7.4.3 Physics-Informed Parameter Bounding

The model has 18 independent trainable parameters across the two GPUs. Unconstrained gradient descent is prone to converging on physically impossible solutions, such as negative thermal mass or heat flowing from cold to hot. To strictly enforce thermodynamic boundaries, all trainable parameters are stored in an unbounded raw space and projected back into their physical ranges during each forward pass using a sigmoid mapping:

$$p = p_{\text{low}} + \sigma(\hat{p}) \cdot \big(p_{\text{high}} - p_{\text{low}}\big) \tag{14}$$

where pˆ is the raw unconstrained *nn.Parameter* value optimized by Adam [15], and p_{low}, p_{high} are the physical bounds. The parameters are initialized by inverting this mapping from their prior values:

$$\widehat{p_{\text{init}}} = \sigma^{-1}\left(\frac{p_{\text{init}} - p_{\text{low}}}{p_{\text{high}} - p_{\text{low}}}\right) = \ln\left(\frac{\text{norm}}{1 - \text{norm}}\right) \tag{15}$$

The description, units, initial values, physical bounds and final optimized values for all 18 parameters are summarized in Table 7.4.

Symbol	Description	Unit	Init	Lower Bound	Upper Bound	Optimized (GPU 0)	Optimized (GPU 1)
C_{die}	Silicon die heat capacity	J/°C	2.0	0.1	10.0	8.9324	8.8707
C_{sink}	Heatsink thermal mass	J/°C	500.0	100.0	5000.0	4713.59	4831.15
R_{paste}	Thermal paste resistance	°C/W	0.05	0.001	0.2	0.0366	0.0336
h_{base}	Passive cooling coeff.	W/°C	4.0	0.5	10.0	3.9791	4.7617
h_{active}	Max active cooling	W/°C	12.0	0.0	30.0	20.9114	19.8178
T_{thresh}	Fan activation temp.	°C	65.0	40.0	85.0	70.2507	67.5134
β	Fan curve steepness	1/°C	0.5	0.05	2.0	1.6688	1.3351
k_{01}	Crosstalk GPU1→GPU0	unitless	0.0780	0.0	0.5	0.0167	—
k_{10}	Crosstalk	unitless	0.0281	0.0	0.5	—	0.0028

	GPU0→GPU1						
q	Ambient heat bias	W	0.0	-10.0	10.0	-8.9202	-8.9428

Table 7.4: ODE Trainable Parameters Description and Values

7.4.4 Training Procedure

The model was set to train for up to 500 epochs utilizing the Adam optimizer. The learning rate is monitored by a *ReduceLROnPlateau* scheduler and was set to 0.01 initially. If the validation RMSE shows no improvement for 3 consecutive epochs, the learning rate is halved. Because the dataset segments vary in their valid sequence lengths, the loss function was defined as a masked Mean Squared Error (MSE). This is computed jointly across both modelled GPUs (GPU 0 and GPU 1) and strictly restricted to valid, non-padded timesteps to prevent zeroes from artificially skewing the loss gradient. The objective function is defined as:

$$\mathcal{L}(\theta) = \frac{E_{\text{error}}}{2\sum_{i=1}^{B}\sum_{t=0}^{S} M_{(i,t)}} \tag{16}$$

where E*error* represents the sum of squared errors across all valid timesteps and modelled GPUs, defined as:

$$E_{\text{error}} = \sum_{i=1}^{B}\sum_{t=0}^{S}\left[\left(\left(\widehat{T_0^{(i,t)}} - T_0^{(i,t)}\right)^2 + \left(\widehat{T_1^{(i,t)}} - T_1^{(i,t)}\right)^2\right)\cdot M_{(i,t)}\right] \tag{17}$$

where T^ die is the predicted silicon die temperature, T_{true} is actual temperature from the dataset, M(i,t) ∈ {0, 1} is the boolean validity mask, B is the batch size, and S is the padded segment length of 5,000 timesteps. The RMSE is computed by taking the square root of the masked MSE L(θ), returning an error in the same units as the temperature (e.g. °C), making it a more interpretable measure of average prediction deviation. RMSE is the metric used to save the best model (least RMSE) during the training process and is defined as:

$$\text{RMSE} = \sqrt{\mathcal{L}(\theta)} \tag{18}$$

The physics-informed ODE model was trained on a workstation equipped with an NVIDIA RTX A4500 accelerator. To efficiently utilize this training hardware across the multi gigabyte dataset, a custom *MultiChunkPrefetcher* was implemented. This daemon thread loaded training chunks from the solid-state drive (SSD) into pinned CPU RAM. To effectively mask I/O latencies, these chunks were streamed to the accelerator's VRAM via non-blocking CUDA streams. To ensure robust generalization, a double-shuffle strategy was applied: training chunks were macro-shuffled prior to each epoch, and the sequences within each chunk were micro-shuffled directly on the compute accelerator. This on device micro-shuffling eliminated severe CPU-to-accelerator synchronization overhead. Furthermore, to prevent repetitive and costly memory swaps, the full validation bundle of 39,533 segments was kept permanently resident in the RTX A4500's VRAM. During the evaluation phase at the end of each epoch, this validation set was processed in batches of 10,000 segments per forward pass to prevent out-of-memory (OOM) failures. Model checkpoints and parameter telemetry logs were saved per epoch, with the final model weights selected based on the strict minimum validation RMSE.

7.5 HPC Cluster Simulator with Integrated Schedulers

To evaluate the real-world efficacy of the calibrated physics-informed ODE, a high-performance JIT compiled digital twin cluster simulator was engineered. While the PyTorch training environment was optimal for backpropagation and parameter calibration, evaluating continuous scheduling policies across thousands of sequential jobs demanded execution speeds that interpreted Python could not provide. Consequently, both the two-mass ODE physics engine (*step_physics_numba*) and the candidate placement logic (*find_best_placement_numba*) were decorated with Numba's @njit(cache=True) directive, compiling both functions to native machine code ahead of the simulation loop. A dedicated JIT warmup pass is executed once at startup to amortize compilation latency before any job is processed.

7.5.1 Job Ingestion and Trace Representation

The simulator ingests historical GPU power traces obtained from the MIT Supercloud dataset, stored as CSV or Parquet files. Each job is represented as a dictionary containing per GPU Welford online [16] accumulators (*mean_0, M2_0, tick_count_0* and their GPU 1 counterparts), per-GPU temperature extrema (*min_temp*, *max_temp*), and a per GPU throttle-step counter. The Welford method was chosen deliberately to compute running mean and

variance without retaining per-tick temperature histories in memory, which would become extensive for large node sweeps. On the first execution over a given input directory, parsed job metadata is serialized to a binary cache file *(_metadata.pkl*) within a subdirectory *(_npy_cache*) of the input path. Power trace arrays are stored as separate .npy binary files per job and GPU. When *PRELOAD_TO_RAM* is enabled, all traces are loaded into a global in-memory dictionary prior to simulation, yielding maximum throughput for datasets that fit within available RAM. When disabled, traces are accessed via NumPy memory-mapped I/O on demand, minimizing RAM footprint for large input sets. The simulator enforces realistic hardware protection throttling logic. When a GPU's die temperature reaches the throttle threshold $T_{throttle}$ = 87.0 °C, the GPU transitions from *STATE_ACTIVE* to *STATE_THROTTLED* and its instantaneous power draw is hard-capped to $P_{throttle}$ = 100 W via *THROTTLE_CAP*, regardless of the workload's actual demand. The GPU remains in this throttled state and the associated job's execution time is extended until passive convection recovers the die temperature below Trecovery = 83.0 °C. Should the die temperature reach the absolute shutdown threshold $T_{shutdown}$ = 90.0 °C, the GPU transitions to STATE_SHUTDOWN, the job is marked as failed, and the node is freed to cool passively. Every elapsed tick during which a GPU is in STATE_THROTTLED is accumulated in the job's *throttledSteps* counter, from which a precise throttle duration in wall-clock seconds is derived at job completion. The simulator supports three operating modes *STANDARD*, *THERMAL_AWARE*, and *AB_TESTING* selectable via the *Config.MODE* field. All physics constants and scheduler thresholds are centralized in *PHYSICS_PARAMS*, and the precomputed inverses of thermal capacitances and paste resistances are cached as module-level constants to avoid redundant division inside the JIT compiled kernel.

7.5.2 Standard Scheduler (FCFS)

To establish a thermally blind control, the Standard scheduler assigns each incoming job to the first available node with sufficient free GPU slots. The placement function scans the *gpu_status* array row by row, assembling candidate *(node, gpu)* pairs into pre-allocated integer arrays *cand_nodes* and *cand_gpus* sized to the maximum possible candidate count, and unconditionally returns the first candidate *(cand_nodes[0])* without consulting any thermal state. This faithfully replicates the first-fit behaviour of conventional HPC resource allocators.

7.5.3 Thermal-Aware Scheduler (Predictive)

To preempt reactive throttling before it occurs, the Thermal Aware scheduler modifies the GPU allocation step with a fast-forward thermal projection executed entirely within the JIT compiled kernel. Prior to any job dispatch, the scheduler copies the current die and heat-sink temperatures of all candidate nodes into a scratch array and runs a forward simulation of N_{proj} physics timesteps, applying a conservative worst-case power draw of $P_{proj} = 250$ W to all GPUs that the current job under consideration would occupy. Neighbouring GPUs on the same node that are currently active or throttled are also projected at P_{proj}, capturing inter-GPU crosstalk via the calibrated k_{01} and k_{10} coefficients. Idle neighbours are held at the baseline idle power draw of $P_{idle} = 25$ W. The projection horizon is set to five simulated minutes:

$$N_{proj} = \left\lfloor \frac{5 \times 60}{\Delta t} \right\rfloor = \left\lfloor \frac{300}{0.11} \right\rfloor = 2727 \text{ timesteps} \tag{19}$$

After projecting all candidate GPUs in a single batched pass, the scheduler selects the candidate whose mean projected thermal stress is minimal. For a single-GPU candidate *i* assigned to GPU $g \in \{0, 1\}$, the stress metric is the time averaged die temperature of that GPU across the horizon. For a dual-GPU candidate, it is the time-averaged per-step maximum across both dies:

$$n^* = \arg\min_{i \in \mathcal{C}} \overline{S^{(i)}} \tag{20}$$

$$\overline{S^{(i)}} = \frac{1}{N_{proj}} \sum_{s=1}^{N_{proj}} \begin{cases} T_{die,g}^{(i,s)}, & \text{if } req\_gpus = 1 \\ \max\left(T_{die,0}^{(i,s)}, T_{die,1}^{(i,s)}\right), & \text{if } req\_gpus = 2 \end{cases} \tag{21}$$

and $T_{die,g}^{(i,s)}$ denotes the projected die temperature of GPU *g* at projection step *s* for candidate *i*. The winning candidate n^* is assigned immediately and the job is dispatched without further gating. Unlike reactive schedulers that respond to throttling events after they occur, this approach uses the physics-informed ODE to make thermally-informed placement decisions proactively at the moment of job dispatch.

7.5.4 Output Artifacts and Archival

Upon completion of each simulation run, a per-run summary CSV and a metadata JSON are

always written, capturing job-level statistics and aggregate fleet-wide metrics respectively. When *EXPORT_GRANULAR_TELEMETRY* is enabled, per-node temperature and power telemetry is additionally written to CSV files using a batched buffer strategy: numerical tuples (t, $T_{die,0}$, $T_{die,1}$, $P0$, $P1$) are accumulated in per-node in-memory buffers and flushed to disk in batched writelines calls whenever a buffer reaches 5,000 entries, with a final flush of any remaining entries after the main loop terminates. When enabled, interactive per-node HTML dashboards embedding Chart.js with pan-and-zoom capability are additionally generated for post-hoc visual analysis. When *HIGH_RES_TELEMETRY* is disabled, telemetry CSV rows are written at every 50th tick (approximately 5.5s intervals). Chart visualization data uses an adaptive rate that scales with job count: every 50th tick for runs with 100 or fewer jobs, every 500th tick (approximately 55s) for up to 500 jobs, and every 5000th tick (approximately 9 minutes) for larger workloads such as the 921-job experiment reported in this paper. Enabling it retains the full $\Delta t = 0.11$ s resolution. All of these artifacts are written to a temporary directory and then compressed into a single ZIP archive, after which the temporary directory is removed.

7.5.5 A/B Testing Mode

The *AB_TESTING* mode does not introduce any new scheduling logic. It is purely a convenience wrapper that executes the Standard scheduler and the Thermal-Aware scheduler back-to-back within a single invocation, feeding each an independent deep copy of the master job queue via *copy.deepcopy(master_queue)*, ensuring complete state isolation between the two runs. Each scheduler produces its own full set of artifacts in an isolated subdirectory. In addition, the A/B run produces a merged side-by-side summary CSV, with Standard and Thermal-Aware per-job columns interleaved, and a combined metadata JSON recording the simulated makespan and aggregate thermal statistics for both policies. This makes a direct per-job comparison immediately available without any post-processing.

7.5.6 Cluster Scaling

The *Config.NODES* parameter accepts a nonempty Python list of integer node counts. The *COOLING_EFFICIENCY_PCT* parameter accepts values in (0, 100] and is applied as a uniform scalar multiplier to the convective cooling coefficients h_{base} and h_{active} for both GPUs, enabling simulation of datacenters with degraded cooling infrastructure without altering any other physical parameter. Regardless of the selected MODE, the simulator iterates over the node list sequentially, executing a complete simulation for each node count in turn. For

example, setting:

```
[*] SYSTEM CONFIGURATION:
    - MODE                      : AB_TESTING
    - NODE(S) TO SIMULATE       : [2, 4, 8, 16, 32, 64, 128, 256]
    - AMBIENT TEMP              : 30 °C
    - COOLING EFFICIENCY        : 100%
```

causes the simulator to execute a full A/B pair at 32 nodes, archive the result, then proceed to 64 nodes, and so on through 256 nodes, all from a single execution. The same sequential sweep is equally valid with *MODE = 'STANDARD'* or *MODE = 'THERMAL_AWARE'*, in which case each tier produces a single-scheduler archive rather than a paired one. This design makes the full scaling study a fire-and-forget operation, the user configures the input once and obtains a structured set of per-tier archives suitable for direct comparative analysis across cluster scales.

7.6 Interactive Web-Based Visualization Dashboard

To provide accessible, real-time visualization User Interface (UI) of the simulator and the scheduler dynamics without requiring a local Python environment, an interactive browser native digital twin was developed and deployed at https://thermal-ode.vercel.app. The implementation required a complete architectural transformation of the Python simulation stack into the browser runtime, preserving numerical accuracy while satisfying the strict concurrency and rendering constraints of the web platform.

7.6.1 TypeScript Physics Port

The two-mass ODE physics engine was fully ported from Python to TypeScript in *physics.ts*. The calibrated parameter set produced by the PyTorch training pipeline is stored as a static JSON file *(calibrated_physics.json)* and imported directly into the frontend module at build time. The combined physics parameter object is assembled at module load time by merging the JSON weights with the fixed simulation timestep $\Delta t = 0.11$ s. The six thermal capacitance and paste-resistance inverses are pre-computed once as module level constants to avoid repeated floating-point division inside the integration loop, in direct correspondence with the Python implementation. The *stepPhysics* function implements the identical Forward Euler integration

and sigmoid-modulated convection model as *step_physics_numba*, operating on a Thermal State object that holds the four state variables $T_{die,0}$, $T_{die,1}$, $T_{sink,0}$, $T_{sink,1}$ for a single node.

7.6.2 CSV Ingestion and Job Parsing

The dashboard accepts MIT Supercloud or any custom trace files directly in the browser via *parser.ts*. Parsing is delegated to the PapaParse library with its native worker: true flag, which automatically offloads the CSV tokenization to a background thread, keeping the main UI thread free during file ingestion. The parsed rows are split by *gpu_index* into two power trace arrays. A file containing rows for both GPU 0 and GPU 1 produces a dual-GPU job *(requested_gpus: 2)*; a file with only one GPU index produces a single-GPU job mapped to power_trace_0. The resulting Job objects carry the same fields as the Python implementation power traces, work-deficit accumulators, per-GPU throttle step counters, and arrival and start timestamps and are forwarded to the simulation Web Worker for scheduling. A pre-parsed quickstart bundle *(quickstart_bundle.json)* is prefetched into browser memory on application mount so that first-time users can launch a demonstration simulation without uploading any files.

7.6.3 Web Worker Simulation Engine

The entire simulation runs in the background using a dedicated Web Worker *(worker.ts)*. This architectural choice ensures that the heavy mathematical computations like the ODE physics integration and continuous scheduler logic happens entirely off the main thread, keeping the user interface smooth and responsive. Rather than relying on the main application to track the simulation, the Web Worker manages the complete *SimulationState* internally and communicates with the UI via a simple messaging system. For example, when the worker receives an INIT setup command, it spins up the virtual server nodes, sets them to the requested ambient temperature, and clears out any old queue data. The main application then passes the batch of uploaded job traces to the worker using an ADD_JOBS command prior to launching the simulation loop. Each call to the internal *tick()* function advances the simulation by one timestep. Within a tick, the worker executes the identical logic established by the Python based simulator.

7.6.4 Master Clock and A/B Lockstep Synchronization

In Standard or Thermal-Aware scheduler mode, the worker drives itself via an internal *setInterval* at the simulation tick rate, executing *stepsPerFrame* ticks per interval according to

the user-selected speed multiplier, and emitting a STATE_DELTA message after each batch. When A/B testing is enabled, two independent worker instances are spawned: Worker A is initialized in Standard mode and Worker B in Thermal-Aware mode. Both receive independent serialized copies of the same job queue with identical arrival timestamps, transmitted as separate ADD_JOBS message payloads to each worker. This achieves the same state isolation as the Python simulator's *copy.deepcopy(master_queue)*, adapted to the structured-clone mechanism of the Web Worker message-passing API. Rather than running on independent internal clocks, both workers are placed in lockstep by a master clock running on the main thread as a *requestAnimationFrame* loop. On each animation frame, the master clock accumulates wall-clock elapsed time against a 110ms or 0.11s tick interval. When sufficient time has accumulated, it sends a *TICK_CHUNK* message carrying the number of ticks to execute to both workers simultaneously, and marks both workers as busy. Neither worker is issued another *TICK_CHUNK* until both have replied with *CHUNK_COMPLETE*, enforcing cycle-exact synchronization between the two simulated universes. This guarantees that the A/B comparison panel displays both schedulers at the same simulated wall-clock instant at all times.

7.6.5 Delta Compression and Rendering Pipeline

To avoid the overhead of serializing and transferring the full simulation state on every tick, the worker maintains internal tracking counters and constructs a *WorkerDelta* message containing only the incremental updates since the previous transmission. The new completed job statistics, chart time labels, and per-node temperature and power arrays are simply appended to the message payload. Instead of forcing the UI to be updated every time a new piece of data arrives, the main application quietly collects these incremental updates in background memory. Visual updates to the dashboard are strictly synchronized with the browser's natural display refresh rate so that its speed would be independent of the computation rate of the underlying calculation engine. This prevents the browser from freezing even under heavy computational load. Even the real-time telemetry graphs are programmed to trigger a render only when a new batch of data explicitly arrives, conserving rendering resources preventing unnecessary chart re-renders.

7.6.6 Data Export

The dashboard supports two export granularities. The sampled export re-uses the down sampled chart data already accumulated in the live buffers (one sample per 50 ticks, approximately 5.5 s intervals) and packages per-node telemetry CSVs together with interactive

HTML dashboards into a ZIP archive for download. The high-resolution export reruns the full simulation silently inside the worker under the *RUN_EXPORT* message handler, logging every tick at the full $\Delta t = 0.11$ s resolution. To avoid exhausting browser heap memory when simulating large clusters across long execution times, the export worker first attempts to acquire a *FileSystemSyncAccessHandle* via the Origin Private File System (OPFS) API. If granted, telemetry rows are flushed to a private on-disk file in chunks of 5000 lines using the synchronous write method, which is permissible inside a Web Worker context. If OPFS access is blocked by the browser, the implementation falls back to in-memory Blob accumulation. Upon completion, the worker transfers the resulting file handles or blobs back to the main thread, which assembles and triggers a ZIP archive download.

Chapter 8

Results and Analysis

The evaluation of this work is partitioned into two sequential phases. First, the predictive accuracy of the calibrated Two Mass ODE is quantified against unseen test dataset. Second, the calibrated ODE is deployed within the cluster simulator to demonstrate its utility as a scheduling policy sandbox, comparing the Standard and Thermal-Aware scheduling policies across node counts ranging from 2 to 256.

8.1 ThermalODE Model Calibration and Predictive Accuracy

8.1.1 Training Convergence and Learning Dynamics:

The Two-Mass ODE was set to train to a maximum of 500 epochs. The best model checkpoint was recorded at epoch 81, at which point the validation RMSE and MAE reached their global minimum of 2.2752 °C and 1.4424 °C respectively. Figure 8.1 presents the training and validation RMSE trajectories and it exhibits no signs of overfitting. The absence of a widening gap or an upward trend in validation error confirms that the model generalizes well to unseen data rather than memorizing the training samples. The ReduceLROnPlateau scheduler halved the learning rate at several points during training, resulting in subsequent improvement of the validation performance before the model ultimately converged.

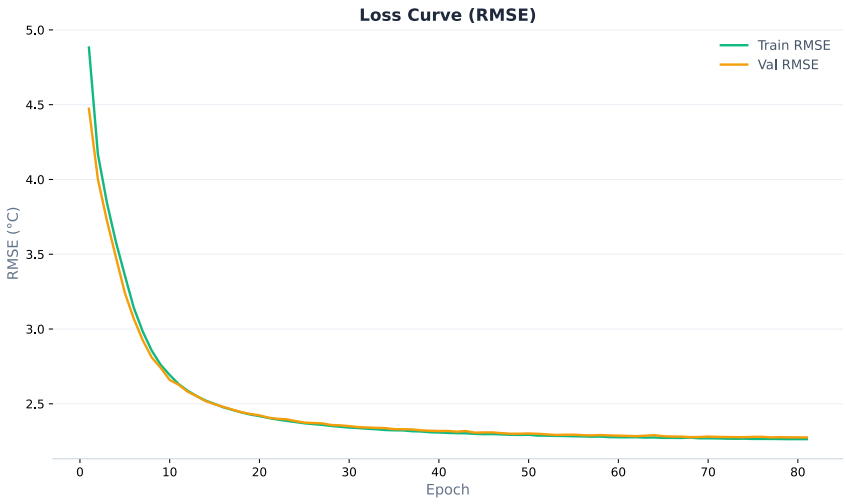

Figure 8.1: Training and validation RMSE curve

8.1.2 Calibrated Physical Parameters:

A critical validation of the physics-informed approach lies not only in predictive accuracy, but in whether the optimized parameter values converge to physically plausible quantities. The final calibrated values, reported in Table 7.4, satisfy this requirement across all 18 parameters. The silicon die capacitances ($C_{die,0}$ = 8.93, $C_{die,1}$ = 8.87 J/°C) are small as physically expected the silicon die has minimal thermal mass and heats rapidly under load. In contrast, the heatsink masses ($C_{sink,0}$ = 4713.6, $C_{sink,1}$ = 4831.2 J/°C) converged near the upper boundary of their search space, consistent with the large heatsink and chassis structures in a server-class GPU node, which act as significant thermal reservoirs. The paste resistances ($R_{paste,0}$ = 0.037, $R_{paste,1}$ = 0.034 °C/W) are physically consistent with standard high-performance thermal interface materials. The dynamic fan curve parameters are notably interpretable. The base convective cooling coefficients ($h_{base,0}$ = 3.98, $h_{base,1}$ = 4.76 W/°C) are appropriately small compared to their active counterparts. These values physically represent the baseline thermal dissipation provided by the server chassis' constant idle airflow before the fans aggressively ramp up, establishing a realistic lower bound for continuous heat removal. Both GPUs calibrated active cooling coefficients as $h_{active,0}$ = 20.91 and $h_{active,1}$ = 19.82, with fan engagement thresholds at $T_{thresh,0}$ = 70.25 °C and $T_{thresh,1}$ = 67.51 °C. The steepness coefficients (β_0 = 1.669, β_1 = 1.335) indicate a sharp fan ramp, reflecting the aggressive step-like behaviour of enterprise level GPU hardware. The crosstalk coefficients reduced significantly during training from their empirically derived initial values. The final calibrated values (k_{01} = 0.0167, k_{10} = 0.0028) are substantially lower than the prior estimates ($\kappa 01$ = 0.0780, $\kappa 10$ = 0.0281). This reduction is physically consistent: the priors were extracted from isolated asymmetric events where one GPU was idle, maximizing observed thermal crosstalk. During normal dual-GPU operation which dominates the training dataset both GPUs are simultaneously active, and the crosstalk signal is largely absorbed into the heatsink's thermal reservoir rather than manifesting as a measurable die temperature rise. The optimizer correctly identified this distinction. Finally, the strongly negative bias values (q0 = −8.92, q1 = −8.94 W) are physically interpretable as a corrective term for the underestimation of convective airflow from the server chassis a heat sink effect that is not captured by the two-GPU-centric model formulation. A striking similarity exists between the calibrated parameter values for GPU 0 and GPU 1. Since these are identical NVIDIA V100 accelerators sitting in the same chassis node, their true physical properties are inherently near-identical. The fact that the gradient descent optimization independently converged on nearly matching physical profiles for both independent GPUs, without being

forced to do so, serves as strong evidence that the true underlying thermodynamics of the hardware has been successfully modelled, rather than relying on arbitrary statistical fits.

8.1.3 Global Predictive Performance:

To quantify generalization capability, evaluation was executed across a separate holdout test set comprising 671 distinct physical jobs, evaluating a total of 443,815,380 timesteps representing thousands of hours of real-world datacenter operation. The ThermalODE achieved a global RMSE of 2.2849 °C and a global MAE of 1.4561 °C. Figure 8.2 presents the density heatmap of predicted versus true temperatures across a 500,000-point random subsample of over 443 million evaluated states, where the concentration of mass along the diagonal (y = x) confirms strong global calibration with minimal systematic offset. A directional bias of +0.1955 °C was measured, indicating a slight systematic tendency to overestimate rather than underestimate temperature. In the context of predictive thermal scheduling, this conservative bias is operationally preferable as an overestimating simulator will trigger thermal-aware interventions fractionally earlier than strictly necessary, whereas an underestimating model risks missing genuine hardware stress events.

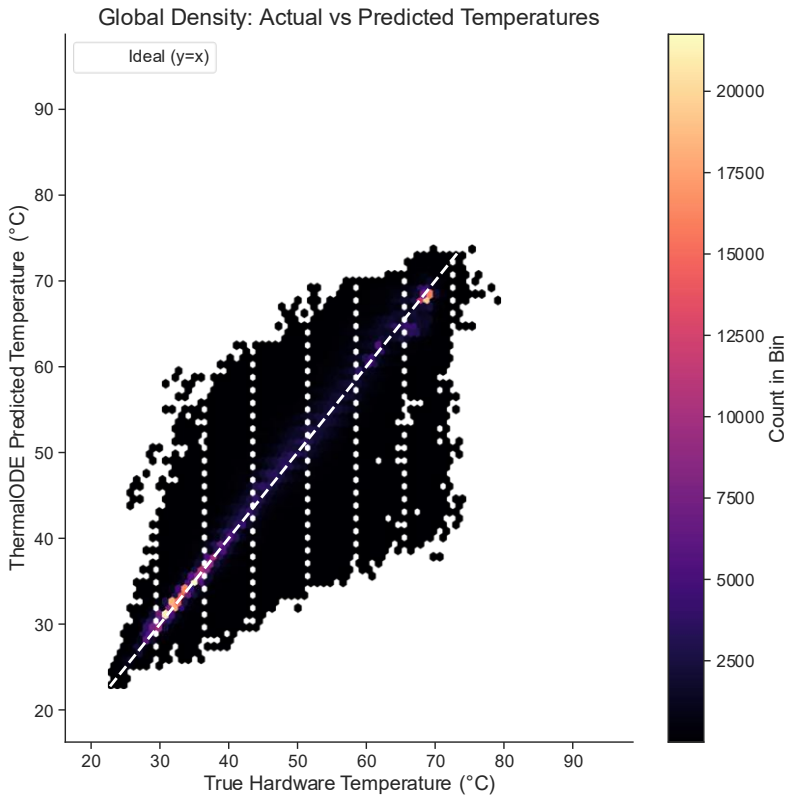

Figure 8.2: Hexbin density heatmap of predicted versus true GPU temperatures

8.1.4 Tail-Risk and Extreme State Evaluation:

For predictive scheduling, the reliability of an ODE based model at its tail is more operationally critical than its median performance. A model that is accurate on average but prone to large error deviations at thermal extremes cannot be safely used for making informed scheduling decisions. Figure 8.3 presents the absolute error distribution across 500,000 randomly subsampled test timesteps. The model maintained an absolute error below 4.66 °C for 95% of all predictions, and below 7.95 °C for 99% of the 443 million evaluated states. The observed maximum error of 34.68 °C occurs in power-step events where the die temperature changes much faster than the Euler-based integrator can keep up with at a $\Delta t = 0.11$ s timestep, a known limitation of explicit first-order integration under steep gradient conditions. An important portion of the evaluation focused exclusively on the Danger Zone, defined as any timestep where the true hardware temperature exceeded 70.0 °C. Within this high stress boundary, the model maintained a RMSE of 5.3853 °C. While this is higher than the global baseline, the physically meaningful point is that the model does not collapse into divergence at the boundary that matters most for scheduling decisions. The sigmoid-gated $h(T_{die})$ formulation correctly captures the nonlinear fan engagement in this regime, as evidenced by the calibrated T_{thresh} values falling within this danger zone (70.25 °C and 67.51 °C for GPU 0 and GPU 1, respectively).

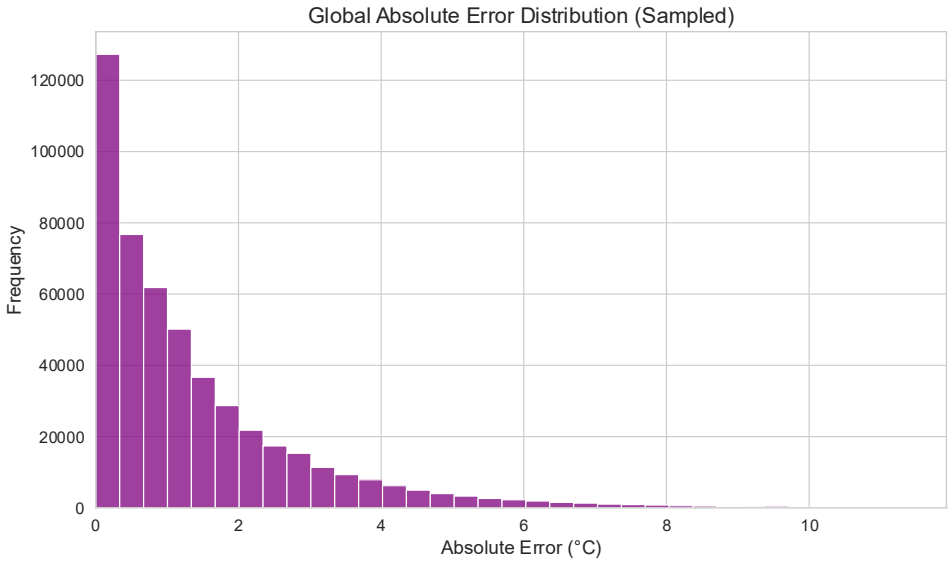

Figure 8.3: Absolute error distribution

8.1.5 Temporal Integration Stability:

A fundamental risk in autoregressive ODE simulation is temporal drift. Prediction errors from early timesteps can accumulate and compound throughout the sequence, causing the model to diverge from physical reality over long workloads. To quantify this, RMSE was computed separately for the first 10% and last 10% of each job's duration, with results shown in Figure 8.4. Counterintuitively, the model demonstrates improving accuracy over the course of a workload. The mean RMSE for the first 10% of job execution is 3.7968 °C, which converges to 2.1676 °C during the final 10%. This behaviour is physically explained by the initialization assumption: the heatsink temperature at t = 0 is not directly observable and is estimated via the thermal equilibrium approximation $T_{sink}(0) = T_{die}(0) - P(0) \cdot R_{paste}$. For jobs that begin with a sudden cold-start power spike, this initialization underestimates the true thermal lag, causing elevated early-stage errors. As the heatsink reaches its true steady-state, the model's predictions converge. The absence of error inflation over prolonged sequences confirms that the physics-informed ODE model successfully constrains the integration within a stable thermodynamic region, unlike unconstrained recurrent networks whose errors diverge on long horizons.

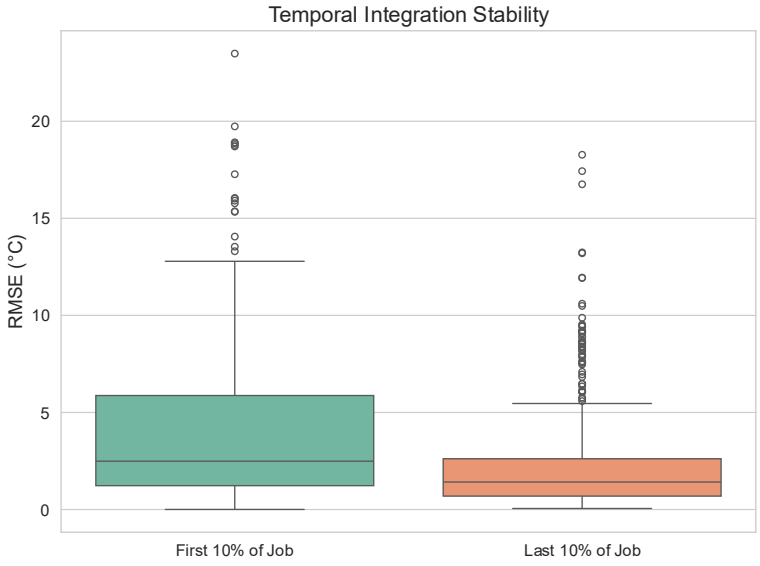

Figure 8.4: RMSE comparison between the first and last 10% of each job

8.2 A/B Testing: Scheduler Policy Evaluation

8.2.1 Experimental Configuration:

To demonstrate the simulator as a practical scheduling policy sandbox, A/B testing was conducted using the full set of 921 heterogeneous GPU job traces sourced from the MIT Supercloud dataset, submitted to both the Standard (FCFS) and Thermal-Aware schedulers

under identical conditions. All simulations used a fixed ambient temperature of 30 °C and nominal cooling efficiency (*COOLING_EFFICIENCY_PCT* = 100), corresponding to the convective parameters calibrated directly from the MIT TX-Gaia cluster hardware. The cluster size was swept across eight tiers: 2, 4, 8, 16, 32, 64, 128, and 256 nodes, with node count as the sole independent variable. No throttling events were observed in either scheduler across any node tier, as temperatures remained below the 87.0 °C hardware threshold throughout. The full aggregated results are reported in Table 8.1.

8.2.2 Simulator Behaviour and Scheduler Convergence:

Table 8.1 shows that under nominal cooling conditions both schedulers produce identical outcomes across all eight node tiers. Makespan, average wait time, average execution time, temperature profiles, and throttle counts are indistinguishable between the Standard and Thermal-Aware policies. This convergence is physically expected and mechanistically consistent with the scheduler implementation. The Thermal-Aware scheduler selects the candidate node with the lowest mean projected thermal stress over a 300-second forward simulation horizon. Under nominal cooling, all idle candidate nodes recover to thermally similar states between consecutive job assignments, leaving no meaningful temperature variance for the projection-based ranking to exploit. The arg min over a set of nearly equal projected temperatures reduces to effectively the same first-available selection as FCFS. The results do confirm several properties of the simulator itself. First, the makespan scales sub-linearly with node count, from 5,598,808 s at 2 nodes to 127,718 s at 256 nodes, a 43.8× reduction, reflecting correct modelling of parallel job execution across the fleet. Second, average execution time remains stable at approximately 24,157 s across all node counts and both schedulers, confirming that job physics are reproduced consistently regardless of placement order. Third, temperatures remain within the range of 64-72 °C across all configurations, consistent with the ODE's calibrated operating regime on V100 hardware and well below the 87.0 °C throttle threshold, validating that the simulator's thermal engine behaves correctly under production workload conditions.

Metric	**2 nodes**	**4 nodes**	**8 nodes**	**16 nodes**	**32 nodes**	**64 nodes**	**128 nodes**	**256 nodes**
Makespan (s)	5,598,808	2,824,896	1,439,132	745,656	405,033	245,630	167,214	127,718

[STD]								
Makespan (s) [TA]	5,598,808	2,824,896	1,439,132	745,656	405,033	245,630	167,214	127,718
Avg Wait Time (s) [STD]	2,712,885	1,345,218	661,169	319,733	149,154	64,133	22,340	4,243
Avg Wait Time (s) [TA]	2,712,885	1,345,218	661,169	319,733	149,154	64,133	22,340	4,243
Avg Exec Time (s) [STD]	24,157	24,157	24,157	24,157	24,157	24,157	24,157	24,157
Avg Exec Time (s) [TA]	24,157	24,157	24,157	24,157	24,157	24,157	24,157	24,157
Absolute Min Temp (°C) [STD]	30	30	30	30	30	30	30	30
Absolute Min Temp (°C) [TA]	30	30	30	30	30	30	30	30
Avg Min Temp (°C) [STD]	57.78	57.73	57.51	56.94	55.94	53.98	50.06	42.19
Avg Min Temp (°C) [TA]	57.92	57.76	57.34	57.02	56.03	54.03	50.1	42.17
Absolute Max Temp	71.60	71.75	71.43	71.81	70.87	70.87	70.88	70.76

(°C) [STD]								
Absolute Max Temp (°C) [TA]	70.84	71.74	71.43	71.81	71.82	70.81	70.88	70.87
Avg Max Temp (°C) [STD]	68.05	68.15	68.17	68.07	68.03	67.97	67.92	67.79
Avg Max Temp (°C) [TA]	68.19	68.15	68.01	68.16	68.11	68.02	67.98	67.78
Mean Temp (°C) [STD]	65.95	66.04	66.02	65.87	65.72	65.5	65.11	64.28
Mean Temp (°C) [TA]	66.1	66.05	65.86	65.96	65.79	65.56	65.15	64.26
Avg Mean Temp (°C) [STD]	65.95	66.04	66.02	65.87	65.72	65.5	65.11	64.28
Avg Mean Temp (°C) [TA]	66.1	66.05	65.86	65.96	65.79	65.56	65.15	64.26
Avg Assignment Temp (°C) [STD]	61.52	61.46	61.2	60.57	59.41	57.17	52.76	43.81
Avg Assignment Temp (°C)	61.65	61.47	61.05	60.65	59.49	57.22	52.8	43.81

[TA]								
Avg Temp Std Dev (°C) [STD]	1.46	1.48	1.53	1.61	1.77	2.06	2.67	3.87
Avg Temp Std Dev (°C) [TA]	1.46	1.48	1.53	1.61	1.77	2.06	2.67	3.87
Throttle Time (s) [STD]	0	0	0	0	0	0	0	0
Throttle Time (s) [TA]	0	0	0	0	0	0	0	0

Table 8.1: A/B Testing Results: 921 Heterogeneous MIT Supercloud Jobs

8.2.3 Limitations and Scope:

The heterogeneous MIT Supercloud workload used in this paper spans 921 real jobs with diverse power profiles and durations, providing a representative sample of production HPC operation under default cooling conditions. Real HPC workloads exhibit heterogeneous power profiles across jobs, which would modulate the relative advantage of the Thermal-Aware policy. Additionally, the simulation fixes the ambient temperature at 30 °C; in practice, datacenter cooling systems themselves respond to thermal load, creating a feedback loop that this simulator does not model. The Thermal-Aware scheduler will gain an edge over the Standard scheduler when high temperatures and throttling events are observed rather than simple placement and queuing decisions under moderate temperatures.

Applications

Datacenter Digital Twin

The validated ThermalODE model provides a digital twin of MIT TX-Gaia cluster nodes that can be used to replay historical workloads, diagnose past thermal incidents, evaluate the thermal impact of novel workload profiles before physical deployment, and benchmark cooling infrastructure improvements. The RMSE of 2.28°C and Mean MAE of 1.45°C on over 443 million unseen timestep confirms sufficient accuracy for deployment as a production digital twin.

Scheduling Policy Sandbox

The A/B testing framework enables researchers to develop and evaluate novel scheduling policies offline using real MIT Supercloud job traces, without access to live cluster hardware. New scheduler implementations can be inserted into the *find_best_placement_numba* function and immediately tested across the full workload suites at configurable cluster scales.

Educational and Research Tool

The browser-based dashboard makes the ThermalODE framework accessible to students and researchers without Python or HPC infrastructure. The quickstart bundle enables immediate demonstration and custom job traces can also be uploaded for institution-specific analysis. The open-source architecture and documented physics model provide a reproducible platform for future HPC thermal research.

Conclusion

Given the increase in GPU dense datacenters, replacing reactive hardware throttling with proactive thermal management is crucial. This paper presented ThermalODE, a physics-informed digital twin and scheduling framework designed to predict and mitigate thermal bottlenecks in HPC clusters. Due to the strict enforcement of thermodynamic boundaries for parameter optimization, the ThermalODE model achieved exceptional predictive accuracy, demonstrating a global RMSE of 2.28 °C across over 443 million evaluated timesteps. The calibrated ODE was deployed within a JIT-compiled cluster simulator capable of evaluating arbitrary scheduling policies against real HPC job traces. A/B testing of a standard FCFS scheduler and a thermal-aware predictive scheduler across 921 heterogeneous MIT Supercloud jobs demonstrated that the simulator correctly reproduces parallel execution scaling and thermal behaviour under production workload conditions, completing all jobs with zero throttling events across all eight node configurations. The simulator's configurability supporting variable node counts, cooling efficiency, and scheduling policies establishes it as a practical sandbox for evaluating thermal management strategies at HPC scale without requiring access to live cluster hardware.

Future Prospects

Room-HVAC and Liquid Cooling Integration

The current model treats ambient temperature as a fixed external boundary condition. Integrating room-level HVAC dynamics into the ODE formulation would enable a complete facility-wide digital twin that captures the feedback loop between cluster thermal load and datacenter cooling system response. Similarly, incorporating liquid-cooling parameters would extend the framework to modern GPU architectures that employ direct-to-chip or immersion cooling.

Dynamic Worst-Case Power Projection

The Thermal-Aware scheduler currently assumes a conservative fixed worst-case power draw of $P_{proj} = 250W$ for all forward projections. A better algorithm that dynamically estimates P_{proj} from the workload's historical power trace signature – or from a learned job-type classifier would reduce false-positive thermal interventions and improve scheduler selectivity under moderate thermal conditions.

CPU Thermal Modelling

The current model explicitly focuses on GPU thermal dynamics. Extending the Two-Mass ODE framework to encompass host CPUs would enable heterogeneous thermal scheduling across both GPU and CPU resources, relevant for workloads that place significant thermal load on both components simultaneously.

Multi-Architecture Generalization

The thermodynamic prior extraction pipeline and Two-Mass ODE architecture are inherently portable. Calibrating the framework for newer GPU architectures such as NVIDIA Ampere (A100), Hopper (H100), Blackwell (B100) and for competing GPU platforms would expand its applicability to the broader HPC and cloud computing ecosystem.

Reinforcement Learning Integration

The ThermalODE simulator provides a differentiable environment that could serve as the training substrate for deep reinforcement learning-based scheduling agents. The physics-

informed ODE model would function as the environment's transition function, enabling RL agents to learn scheduling policies that optimize for complex multi-objective criteria (makespan, thermal safety, energy efficiency) through interaction with the digital twin rather than the live cluster.

Real-Time Production Deployment

The immediate next step in deployment is integration with a production SLURM instance. The Numba-compiled ODE can be served as a low-latency microservice that SLURM queries via a Lua or Python SLURM plugin at job dispatch time. Long-term, continuous online re-calibration updating ODE parameters from streaming live telemetry using sequential gradient updates would allow the digital twin to adapt to hardware ageing, dust accumulation in cooling systems, and changing workload profiles over the operational lifetime of the cluster.

References

[1] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux Utility for Resource Management," in Job Scheduling Strategies for Parallel Processing, Springer, 2003, pp. 44–60.

[2] NVIDIA Corporation, "NVIDIA Tesla V100 GPU Architecture," NVIDIA Whitepaper, 2017.

[3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," arXiv:1912.01703, 2019.

[4] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-Based Python JIT Compiler," in Proc. LLVM '15, ACM, 2015.

[5] S. Go et al., "Characterizing the Efficiency of Distributed Training: A Power, Performance, and Thermal Perspective," arXiv:2509.10371, 2025.

[6] Y. Lee, K. G. Shin, and H. S. Chwa, "Thermal-Aware Scheduling for Integrated CPUs-GPU Platforms," ACM Trans. Embed. Comput. Syst., vol. 18, no. 5s, Oct. 2019.

[7] B. Kocot, P. Czarnul, and J. Proficz, "Energy-Aware Scheduling for High-Performance Computing Systems: A Survey," Energies, vol. 16, no. 2, 2023.

[8] Y. Liu, P. S. Shah, T. Xia, and D. Huston, "Self-Referencing Digital Twin for Thermal and Task Management," Computers, vol. 15, no. 1, 2026.

[9] J. Ran et al., "Co-Optimization of Thermal-Aware Workload Scheduling with Deep RL-Based Cooling Control in Data Centers," Energy, vol. 344, p. 139965, 2026.

[10] L. Di Natale, B. Svetozarevic, P. Heer, and C. Jones, "Physically Consistent Neural Networks for Building Thermal Modeling," Applied Energy, vol. 325, p. 119806, 2022.

[11] M. Maiterth et al., "HPC Digital Twins for Evaluating Scheduling Policies, Incentive Structures and Their Impact on Power and Cooling," in Proc. SC '25 Workshops, ACM, Nov. 2025, pp. 1959–1969.

[12] J. Stojkovic et al., "TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms," arXiv:2501.02600, 2025.

[13] Y. Zhang et al., "A Real-Time Digital Twin for Adaptive Scheduling," arXiv:2512.18894, 2025.

[14] S. Samsi et al., "The MIT Supercloud Dataset," in Proc. IEEE HPEC 2021, pp. 1–8.

[15] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv:1412.6980, 2017.

[16] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," Technometrics, vol. 4, no. 3, pp. 419–420, 1962.

Publication Details

Research Paper

Title: ThermalODE: A Physics-Informed Simulator for Multi-GPU HPC Clusters

Authors: Saket Sontakke, Taha Kapadia, Pranjal Vishwakarma, Akshat Srivastava

Venue: IEEE Transactions on Parallel and Distributed Systems (TPDS)

Status: Manuscript in preparation for submission to IEEE TPDS

Abstract: The increase in Artificial Intelligence (AI) and Deep Learning (DL) workloads has resulted in exponential growth in computational demands within High-Performance Computing (HPC) clusters. To satisfy these requirements, modern HPC cluster nodes are composed of dense, multi-GPU accelerated processors. However, this concentration of processors generates a significant amount of heat, elevating node temperatures to levels where the hardware is adversely affected or, in worse cases, enters a throttled state to prevent damage. During this throttled state, clock speeds are reduced, degrading the computational power of the processor and the overall efficiency of the cluster. This paper presents ThermalODE, an Ordinary Differential Equations (ODE) based physics-informed model trained on real-world telemetry from the MIT Supercloud Dataset to simulate the thermal behaviour of individual nodes. The model was evaluated on over 443 million unseen timesteps across 671 jobs, and achieved a global Root Mean Square Error (RMSE) of 2.28 °C and Mean Absolute Error (MAE) of 1.45°C, validating its use as a digital twin for the cluster. This digital twin is then deployed within a JIT-compiled cluster simulator to demonstrate its utility as a scheduling policy sandbox. A/B testing of a standard First Come First Serve (FCFS) scheduler against a thermal-aware predictive scheduler was conducted across 921 heterogeneous MIT Supercloud job traces at eight node scales (2 to 256 nodes) under default cooling conditions. Both schedulers successfully completed all 921 jobs with zero throttling events, with makespan scaling from 5,598,808 seconds at 2 nodes to 127,718 seconds at 256 nodes, a 43.8× reduction, validating the simulator's correct modelling of parallel execution and thermal physics under production workload conditions.

Appendices

Appendix A: Acronyms and Abbreviations

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CFD	Computational Fluid Dynamics
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DL	Deep Learning
DVFS	Dynamic Voltage and Frequency Scaling
FCFS	First-Come-First-Serve
GPU	Graphics Processing Unit
HPC	High-Performance Computing
HVAC	Heating, Ventilation and Air Conditioning
JIT	Just-In-Time
JSON	JavaScript Object Notation
LLM	Large Language Model
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error
ODE	Ordinary Differential Equations
OOM	Out of Memory
OPFS	Origin Private File System
RAM	Random Access Memory
RL	Reinforcement Learning
RMSE	Root Mean Square Error

SLURM Simple Linux Utility for Resource Management

SSD Solid-State Drive

UI User Interface

UML Unified Modeling Language

VRAM Video Random Access Memory

Appendix B: Model Parameter Evolution Graphs